

# 算法设计与分析

## 第6章 分支限界法 (Branch and Bound)



- 理解分支限界法的剪枝搜索策略
- 掌握分支限界法的算法框架
  - (1) 队列式(FIFO)分支限界法
  - (2) 优先队列式分支限界法
- 通过应用范例学习分支限界法的设计策略
  - (1) 单源最短路径问题 (2) 装载问题 (3) 0-1背包问题
  - (4) 最大团问题 (5) 旅行售货员问题 (6) 批处理作业调度问题

# 6.1 分支限界法的基本思想



## 分支限界法与回溯法

- 求解目标：回溯法的求解目标是找出解空间树中满足约束条件的所有解，而分支限界法的求解目标则是找出满足约束条件的一个解，或是在满足约束条件的解中找出在某种意义下的最优解。
- 搜索方式的不同：回溯法以深度优先的方式搜索解空间树，而分支限界法则以广度优先或以最小耗费优先的方式搜索解空间树。

# 6.1 分支限界法的基本思想



## 分支限界法与回溯法

- 分支限界法常以广度优先或以最小耗费（最大效益）优先的方式搜索问题的解空间树。
- 在分支限界法中，每一个活结点只有一次机会成为扩展结点。活结点一旦成为扩展结点，就一次性产生其所有儿子结点。在这些儿子结点中，导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被加入活结点表中。
- 此后，从活结点表中取下一结点成为当前扩展结点，并重复上述结点扩展过程。这个过程一直持续到找到所需的解或活结点表为空时为止。

# 6.1 分支限界法的基本思想



## 常见的两种分支限界法

### (1) 队列式 (FIFO) 分支限界法

按照队列先进先出 (FIFO) 原则选取下一个节点为扩展节点。

数据结构：队列

### (2) 优先队列式分支限界法

按照优先队列中规定的优先级选取优先级最高的节点成为当前扩展节点。

数据结构：堆

最大优先队列：最大堆，体现最大效益优先

最小优先队列：最小堆，体现最小费用优先

# 6.1 分支限界法的基本思想



## 队列式 (FIFO) 分支限界法

- 搜索策略：一开始，根结点是唯一的活结点，根结点入队。从活结点队中取出根结点后，作为当前扩展结点。对当前扩展结点，先从左到右地产生它的所有儿子，用约束条件检查，把所有满足约束函数的儿子加入活结点队列中。再从活结点表中取出队首结点为当前扩展结点，……，直到找到一个解或活结点队列为空为止。

# 6.1 分支限界法的基本思想



## 优先队列式分支限界法

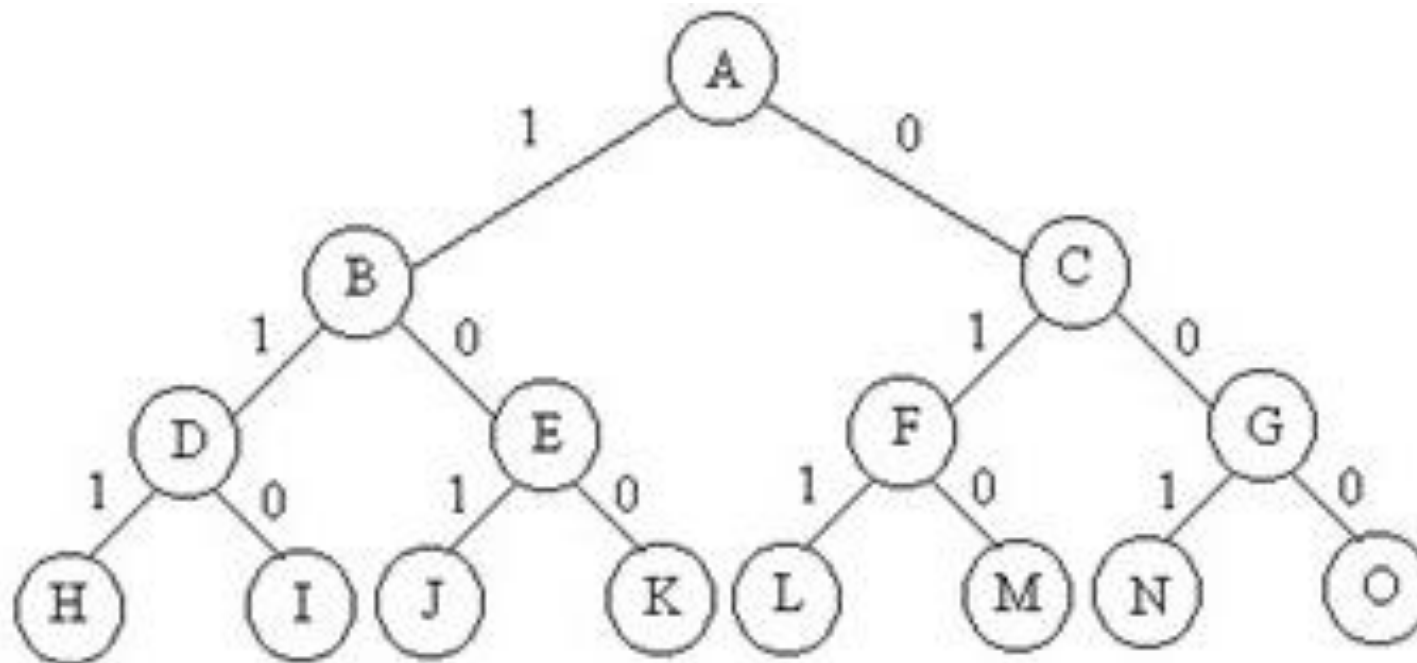
- 搜索策略：对每一活结点计算一个优先级（某些信息的函数值），并根据这些优先级；从当前活结点表中优先选择一个优先级最高（最有利）的结点作为扩展结点，使搜索朝着解空间树上有最优解的分支推进，以便尽快地找出一个最优解。再从活结点表中下一个优先级别最高的结点为当前扩展结点，……，直到找到一个解或活结点队列为空为止。

# 6.1 分支限界法的基本思想



## 分支限界法搜索应用举例

- 0-1背包问题，当 $n=3$ 时， $w=\{16,15,15\}$ ， $p=\{45,25,25\}$ ， $c=30$

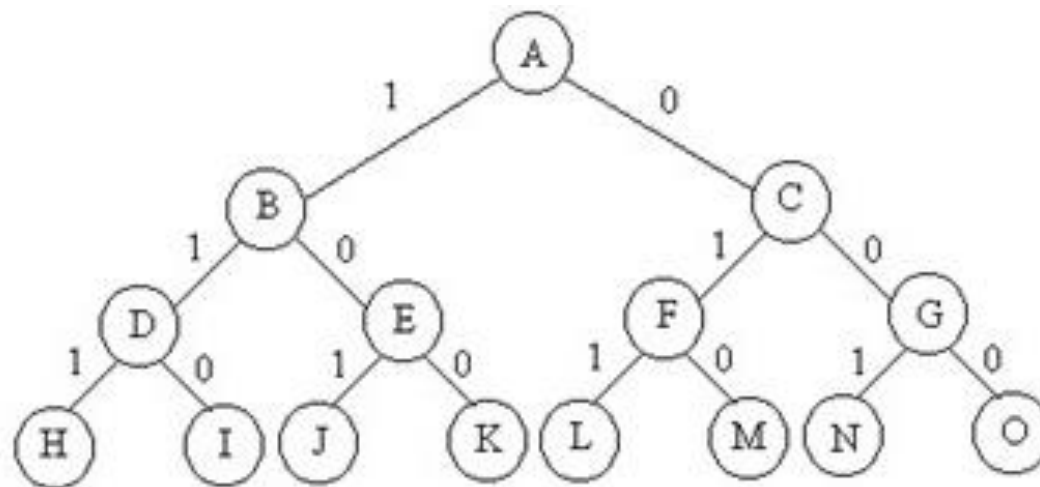


# 6.1 分支限界法的基本思想



## 分支限界法搜索应用举例

- 0-1背包问题，当 $n=3$ 时， $c=30$ ， $w=\{16,15,15\}$ ， $p=\{45,25,25\}$



解空间广度搜索

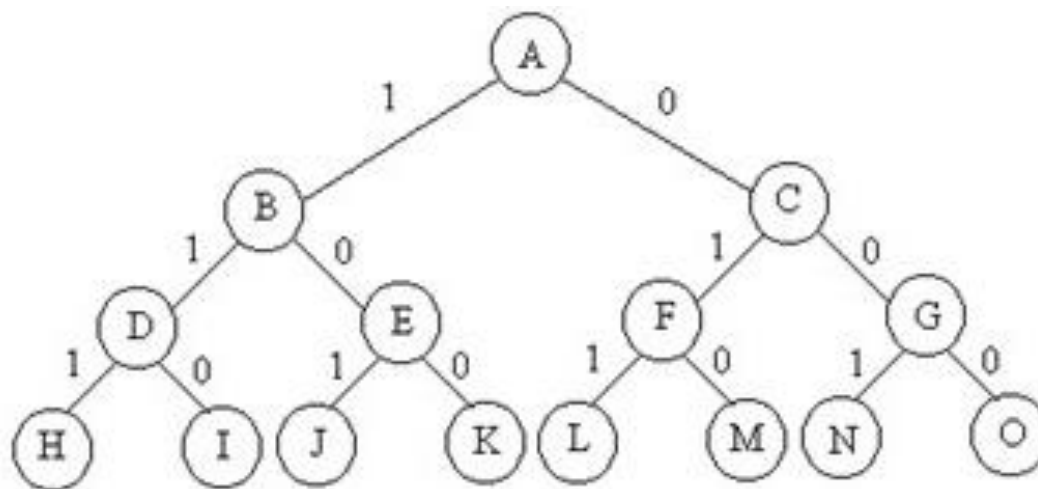
- 队列式分支限界法(处理法则:先进先出):  $\{\} \rightarrow \{A\} \rightarrow \{B,C\} \rightarrow \{C,D,E\}$  (D是不可行解, 舍弃)  $\rightarrow \{C,E\} \rightarrow \{E,F,G\} \rightarrow \{F,G,J,K\}$  (J是不可行解, 舍弃)  $\rightarrow \{F,G,K\} \rightarrow \{G,K,L,M\} \rightarrow \{K,L,M,N,O\} \rightarrow \{\}$

# 6.1 分支限界法的基本思想



## 分支限界法搜索应用举例

- 0-1背包问题，当 $n=3$ 时， $c=30$ ， $w=\{16,15,15\}$ ， $p=\{45,25,25\}$



解空间广度搜索

- 优先队列式分支限界法(处理法则:价值大者优先):  $\{\} \rightarrow \{A\} \rightarrow \{B(45), C(0)\} \rightarrow \{C(0), D(x), E(45)\} \rightarrow \{C(0), E(45)\} \rightarrow \{C(0), J(x), K(45)\} \rightarrow \{C\} \rightarrow \{F(25), G(0)\} \rightarrow \{G(0), L(50), M(25)\} \rightarrow \{G(0), M(25)\} \rightarrow \{G\} \rightarrow \{N, O\} \rightarrow \{O\} \rightarrow \{\}$

# 6.1 分支限界法的基本思想



## 算法步骤

- (1) 定义解空间
- (2) 确定易于搜索的解空间结构（二叉树。。）
- (3) 确定结点的数据结构
- (4) 确定约束条件与限界条件用于剪枝
- (5) 确定活结点组织方法（队列/优先队列（优先级函数））
- (6) 确定答案节点识别方法

## 6.4 0-1背包问题

### 优先队列分支限界改进算法思想

- 首先，要对输入数据进行预处理，将各物品依其单位重量价值从大到小进行排列。
- 在下面描述的优先队列分支限界法中，节点的优先级由已装袋的物品价值加上剩下的最大单位重量价值的物品装满剩余容量的价值和。
- 算法首先检查当前扩展结点的左儿子结点的可行性。如果该左儿子结点是可行结点，则将它加入到子集树和活结点优先队列中。当前扩展结点的右儿子结点一定是可行结点，仅当右儿子结点满足上界约束时才将它加入子集树和活结点优先队列。当扩展到叶节点时为问题的最优值。

## 6.4 0-1背包问题

- 用优先队列式分枝限界法解决0/1背包问题，要确定以下四个问题：
  - 解空间树中结点的结构；
  - 如何生成一个给定结点的儿子；
  - 如何组织活结点表；
  - 如何识别答案结点。

## 6.4 0-1背包问题

### 解空间树中结点的结构

- 采用完全二叉树作为解空间树，每个结点包含：
  - Parent: 结点X的父亲结点连接指针;
  - Level: 结点X在解空间树中的级数，置 $X_{Level(X)} = 1$ 表示生成X的左儿子，置 $X_{Level(X)} = 0$ 表示生成X的右儿子;
  - Cleft: 记录背包在结点X处的可用空间(即剩余空间)，在确定X左儿子的可行性时用;
  - Cp: 记录在结点X处背包中已装物品的价值(或效益值)， $Cp = \sum_{1 \leq i < Level(X)} p_i x_i$ 。

## 6.4 0-1背包问题

### 如何生成儿子（判断可行性）

- 检查当前扩展结点的左儿子结点的可行性：  
如果该左儿子结点满足约束条件，则它是可行的，将它加入到子集树和活结点优先队列中。
- 检查当前扩展结点的右儿子结点的可行性：  
右儿子一定是可行结点（满足约束条件），仅当右儿子结点满足限界时才将它加入子集树和活结点优先队列。
- 当扩展到叶节点时为问题的最优值。

## 6.4 0-1背包问题

### 如何生成儿子（判断限界条件）

- 约束条件:  $cw + wk \leq C$ , 即  $wk \leq Cleft$
  - 限界条件: 在结点X所表示的状态下, 可行解所能达到的可能值的上界用bound表示。也即是说, 当当前结点的值确定后, 可行解所能达到的效益值的上界。
  - 如果到目前为止所知道的解的最大效益值为bestc, 当 $bound(X) < bestc$ 时, 就应该处死结点X。所以,  $bound(X)$ 可以作为限界函数。
- 
- Cleft: 记录背包在结点X处的可用空间(即剩余空间), 在确定X左儿子的可行性时用;
  - Cp: 记录在结点X处背包中已装物品的价值(或效益值),
  - $bound(X)$ =当前背包价值量+剩余物品按最大单位价值依次装入直至装满背包

## 6.4 0-1背包问题

### 如何组织活节点表

- 即：如何安排活结点的优先顺序。
- 定义的限界函数 $\text{bound}(X)$ 为当前剩余最大单位价值的物品装满背包时的背包价值，也就是说 $\text{bound}(X)$ 越大，该子树中产生最优解的可能越大。所以， $\text{bound}(X)$ 也是优先级函数。
- $\text{bound}(X)$  = 当前背包价值量 + 剩余物品按最大单位价值依次装入直至装满背包
- 每个活结点的优先顺序由 $\text{bound}(X)$ 来确定。

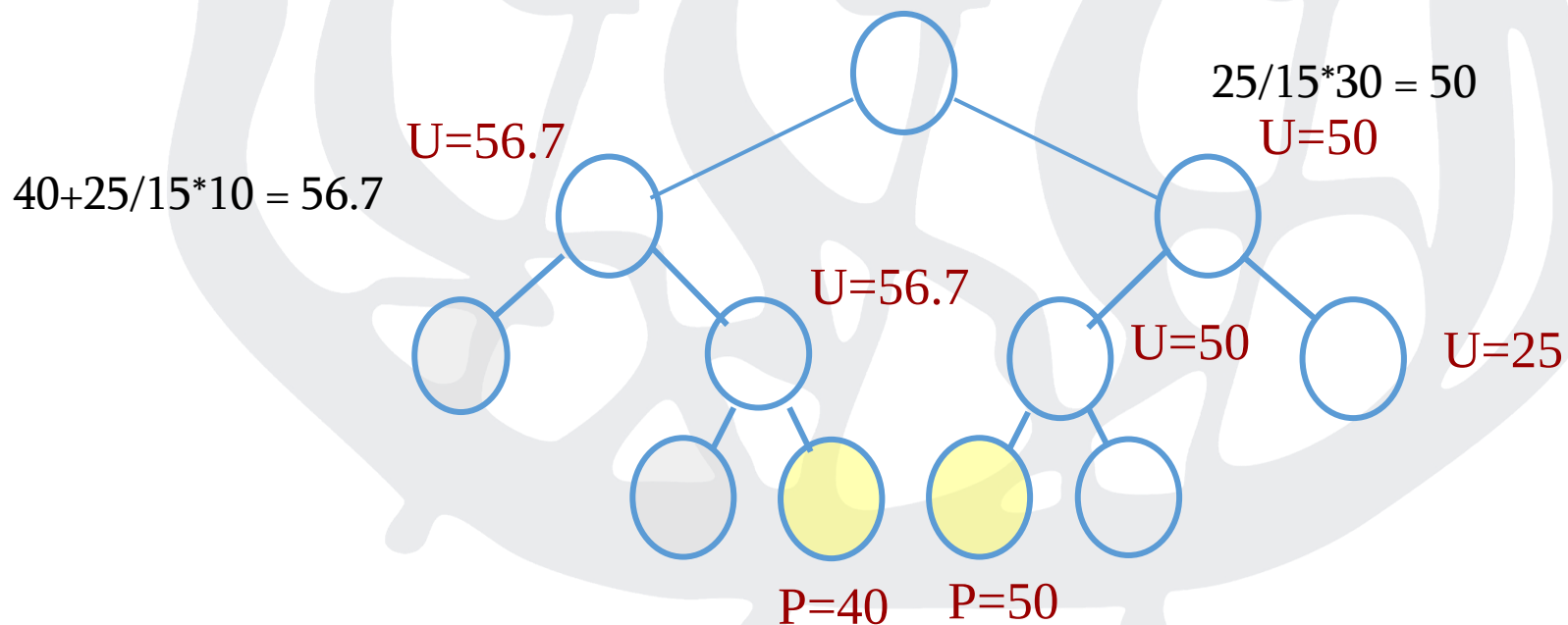
## 6.4 0-1背包问题

### 如何识别答案节点

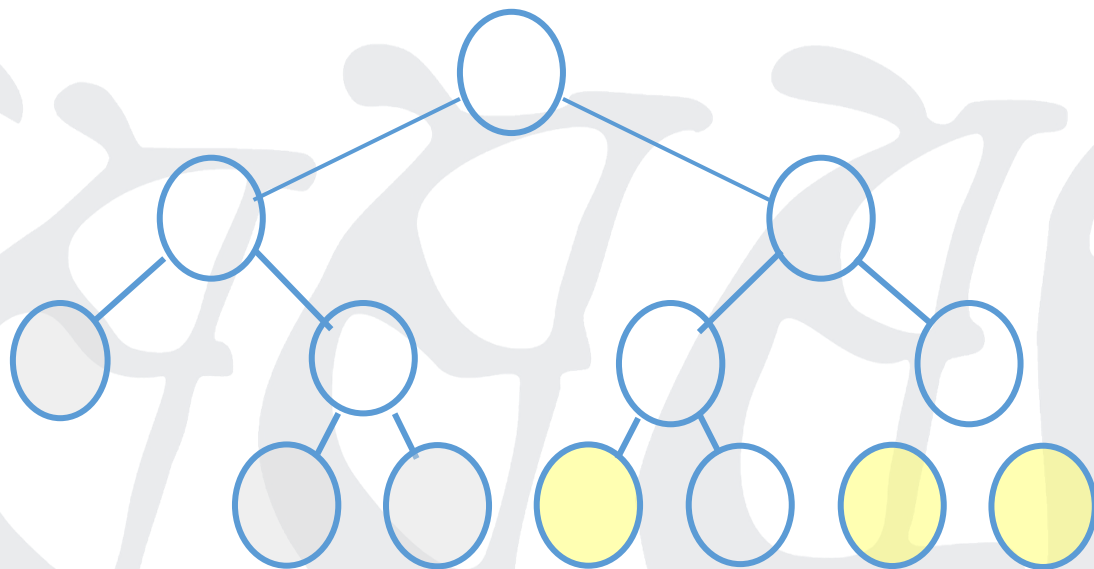
- 答案结点指使得背包价值最大的物品序列集合。
- 当搜索到的叶子结点的cp值大于所有活结点列表中结点的bound的时候或者活结点列表为空时找到最优值。将对应的X[]序列输出即为最优解。
- Cleft: 记录背包在结点X处的可用空间(即剩余空间), 在确定X左儿子的可行性时用;
- Cp: 记录在结点X处背包中已装物品的价值(或效益值),

## 6.4 0-1背包问题

- 设 $N=3$ ,  $W=(20,15,15)$ ,  $P=(40,25,25)$ ,  $C=30$ 。把 $Ubound=$ “当前背包价值量+剩余物品按最大单位价值依次装入直至装满背包”作为优先级。

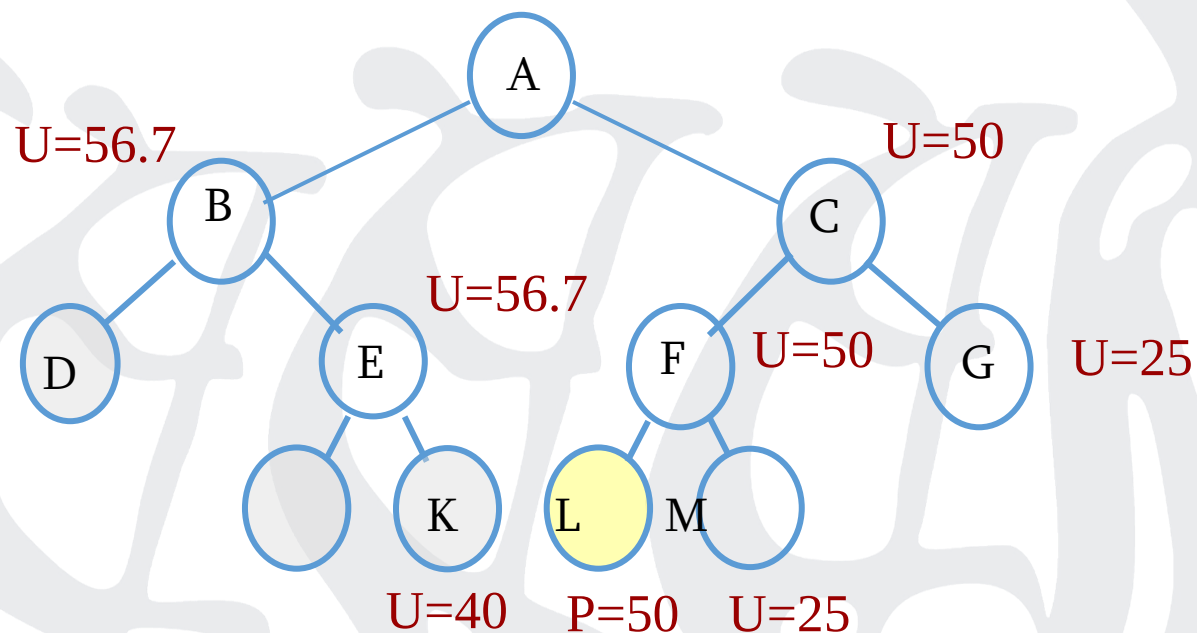


## 6.4 0-1背包问题——队列遍历



- 队列：运行到叶子结点时记录该节点的价值
- $\{A\} \rightarrow \{B,C\} \rightarrow \{C,E\} \rightarrow \{E,F,G\} \rightarrow \{F,G,K\} \rightarrow \{G,K,L,M\} \rightarrow \{K,L,M,N,O\} \rightarrow \{L,M,N,O\} \rightarrow \{M,N,O\} \rightarrow \{N,O\} \rightarrow \{O\} \rightarrow$  最优L结点

## 6.4 0-1背包问题



- 优先队列:  $\{A\} \rightarrow \{B, C\} \rightarrow \{E, C\} \rightarrow \{C, K\} \rightarrow \{F, K, G\} \rightarrow \{L, K, G, M\}$
- L结点为叶子节点, 最优

## 6.4 0-1背包问题

### 算法描述

初始化, 将物品按照单位价值从大到小的顺序排列  
从第一个物品开始

左儿子加入后可否可满足控制约束

将顶点 $i$ 加入到最大优先队列中

修改最大价值, 修改上界条件

检查右儿子可否满足条件

将右儿子加入优先队列

修改上界条件

取下一扩展结点

当队列为空时

输出当前记录的路径

"取头、生子、  
过滤、入尾"

## 6.4 0-1背包问题

### ■ 上界函数Bound

```
while (i <= n && w[i] <= cleft) {    // n表示物品总数, cleft为剩余空间
    cleft -= w[i];                    // w[i]表示i所占空间
    b += p[i];                        // p[i]表示i的价值
    i++;
}
if (i <= n) b += p[i] / w[i] * cleft; // 装填剩余容量装满背包
return b;                            // b为上界函数
```

## 6.4 0-1背包问题

```
while (i != n+1) { // 非叶结点
    // 检查当前扩展结点的左儿子结点
    Typew wt = cw + w[i];
    if (wt <= c) { // 左儿子结点为可行结点
        if (cp+p[i] > bestp) bestp = cp+p[i];
        AddLiveNode(up, cp+p[i], cw+w[i], true, i+1);
        up = Bound(i+1);
        // 检查当前扩展结点的右儿子结点
        if (up >= bestp) // 右子树可能含最优解
            AddLiveNode(up, cp, cw, false, i+1);
        // 取下一个扩展节点 (略)
    }
}
```

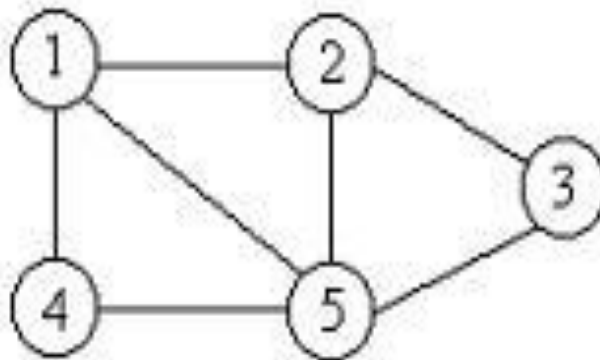
分支限界搜索过程

## 6.5 最大团问题



### 1. 问题描述

- 给定无向图 $G=(V, E)$ 。如果 $U \subseteq V$ ，且对任意 $u, v \in U$ 有 $(u, v) \in E$ ，则称 $U$ 是 $G$ 的完全子图。 $G$ 的完全子图 $U$ 是 $G$ 的团当且仅当 $U$ 不包含在 $G$ 的更大的完全子图中。 $G$ 的最大团是指 $G$ 中所含顶点数最多的团。
- 下图 $G$ 中，子集 $\{1, 2\}$ 是 $G$ 的大小为2的完全子图。这个完全子图不是团，因为它被 $G$ 的更大的完全子图 $\{1, 2, 5\}$ 包含。 $\{1, 2, 5\}$ 是 $G$ 的最大团。 $\{1, 4, 5\}$ 和 $\{2, 3, 5\}$ 也是 $G$ 的最大团。



### 2. 上界函数

- 用变量cliqueSize表示与该结点相应的团的顶点数；level表示结点在子集空间树中所处的层次；用 $\text{cliqueSize} + n - \text{level} + 1$ 作为顶点数上界upperSize的值。
- 在此优先队列式分支限界法中，upperSize实际上也是优先队列中元素的优先级。算法总是从活结点优先队列中抽取具有最大upperSize值的元素作为下一个扩展元素。

### 3. 算法思想

- 子集树的根结点是初始扩展结点，对于这个特殊的扩展结点，其cliqueSize的值为0。
- 算法在扩展内部结点时，首先考察其左儿子结点。在左儿子结点处，将顶点 $i$ 加入到当前团中，并检查该顶点与当前团中其它顶点之间是否有边相连。当顶点 $i$ 与当前团中所有顶点之间都有边相连，则相应的左儿子结点是可行结点，将它加入到子集树中并插入活结点优先队列，否则就不是可行结点。
- 接着继续考察当前扩展结点的右儿子结点。当 $upperSize > bestn$ 时，右子树中可能含有最优解，此时将右儿子结点加入到子集树中并插入到活结点优先队列中。

### 3. 算法思想

- 算法的while循环的终止条件是遇到子集树中的一个叶结点(即 $n+1$ 层结点)成为当前扩展结点。
- 对于子集树中的叶结点, 有 $\text{upperSize} = \text{cliqueSize}$ 。此时活结点优先队列中剩余结点的 $\text{upperSize}$ 值均不超过当前扩展结点的 $\text{upperSize}$ 值, 从而进一步搜索不可能得到更大的团, 此时算法已找到一个最优解。

## 6.5 最大团问题



//解最大团问题的优先队列式分支限界法

```
int Clique::BBMaxClique(int bestx[]) {  
    MaxHeap<CliqueNode> H(1000);  
    //初始化  
    bbnode *E = 0;  
    int i = 1,  
        cn = 0,  
        bestn = 0;  
    //搜集子集空间树  
    while(i!=n+1)//非叶节点 {  
        //检查顶点i与当前团中其他顶点之间是否有边相连  
        bool OK = true;  
        bbnode *B = E;  
        for(int j=i-1; j>0; B=B->parent,j--) {  
            if(B->LChild && a[i][j]==0) {  
                OK = false;  
                break;  
            }  
        }  
        if(OK) {  
            bbnode *N = new bbnode;  
            N->parent = B;  
            N->LChild = i;  
            H.Insert(N);  
            if(N->LChild == n) {  
                bestn = bestx[n];  
            }  
            E = N;  
            i++;  
        }  
    }  
    return bestn;  
}
```

## 6.5 最大团问题



```
if(OK) { // 左儿子节点为可行结点
    if(cn+1>bestn) bestn = cn + 1;
    AddLiveNode(H,cn+1,cn+n-i+1,i+1,E,true);}

if(cn+n-i>=bestn) { // 右子树可能含有最优解
    AddLiveNode(H,cn,cn+n-i,i+1,E,false); }

// 取下一扩展节点
CliqueNode N;
H.DeleteMax(N); // 堆非空
E = N.ptr;
cn = N.cn;
i = N.level;
}

// 构造当前最优解
for(int j=n; j>0; j--)
{
    bestx[j] = E->LChild;
    E = E->parent;
}

return bestn;
}
```

## 6.6 旅行售货员问题



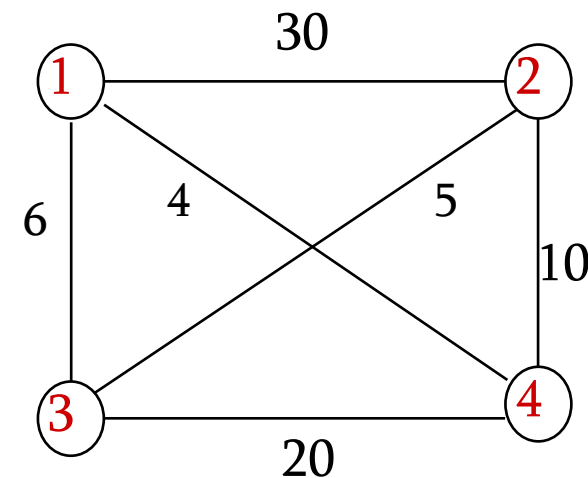
### 1. 问题描述

- 某售货员要到若干城市去推销商品，已知各城市之间的路程(或旅费)。他要选定一条从驻地出发，经过每个城市一次，最后回到驻地的路线，使总的路程(或总旅费)最小。
- 路线是一个带权图。图中各边的费用(权)为正数。图的一条周游路线是包括 $V$ 中的每个顶点在内的一条回路。周游路线的费用是这条路线上所有边的费用之和。
- 旅行售货员问题的解空间可以组织成一棵树，从树的根结点到任一叶结点的路径定义了图的一条周游路线。旅行售货员问题要在图 $G$ 中找出费用最小的周游路线。
- 这里可以用优先队列式实现方式。

## 6.6 旅行售货员问题



- 算法开始时创建一个最小堆，用于表示活结点优先队列。(用一个最小优先队列来记录活节点)，队列中每个节点的类型为MinHeapNode。
- 每个结点中包括：
  - $x[0:n-1]$ : 解的结构。
  - $s$ : 表示结点在排列树中的层次
  - $cc$ : 解空间树中从根节点到当前节点的耗费。
  - $rcost$ 为 $x[s:n-1]$ 中(当前顶点加上剩余所有顶点)顶点最小出边费用和。图中每个顶点的最小出边费用用 $minout$ 记录。
  - $lcost$ : 该节点子树的最小耗费下界。
  - $Bound=lcost=cc+rcost$ 为当前结点的下界。



|   | 1        | 2        | 3        | 4        |
|---|----------|----------|----------|----------|
| 1 | $\infty$ | 30       | 6        | 4        |
| 2 | 30       | $\infty$ | 5        | 10       |
| 3 | 6        | 5        | $\infty$ | 20       |
| 4 | 4        | 10       | 20       | $\infty$ |

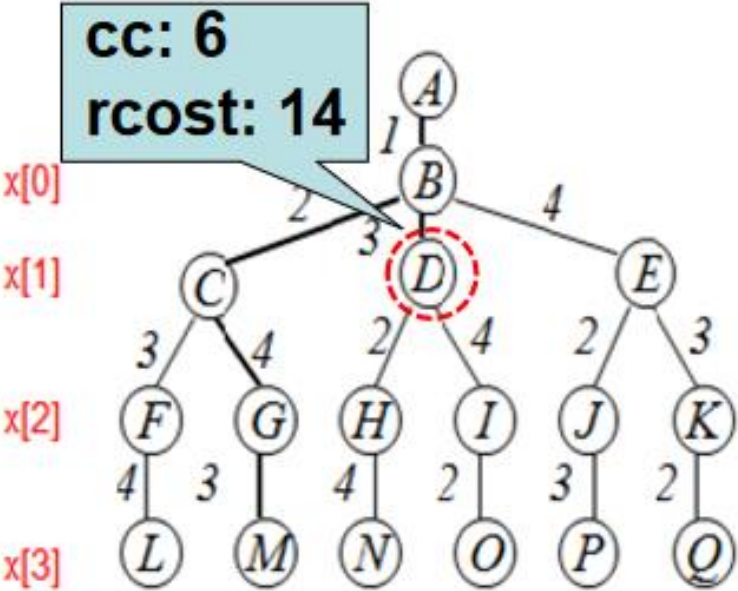
最小出边费用Minout

Minout[1]=4 Minout[2]=5

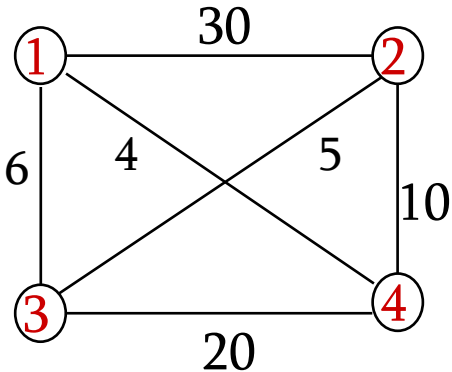
Minout[3]=5 Minout[4]=4

# 6.6 旅行售货员问题

- 每个结点中包括：
  - s: 表示结点在排列树中的层次
  - cc: 解空间树中从根节点到当前节点的耗费。
  - lcost: 该节点子树的最小耗费下界。
  - rcost为x[s:n-1]中(当前顶点加上剩余所有顶点)顶点最小出边费用和。图中每个顶点的最小出边费用用minout记录。
  - Bound=lcost=cc + rcost为当前结点的下界。



|   | 1        | 2        | 3        | 4        |
|---|----------|----------|----------|----------|
| 1 | $\infty$ | 30       | 6        | 4        |
| 2 | 30       | $\infty$ | 5        | 10       |
| 3 | 6        | 5        | $\infty$ | 20       |
| 4 | 4        | 10       | 20       | $\infty$ |



Minout[1]=4 Minout[2]=5  
Minout[3]=5 Minout[4]=4

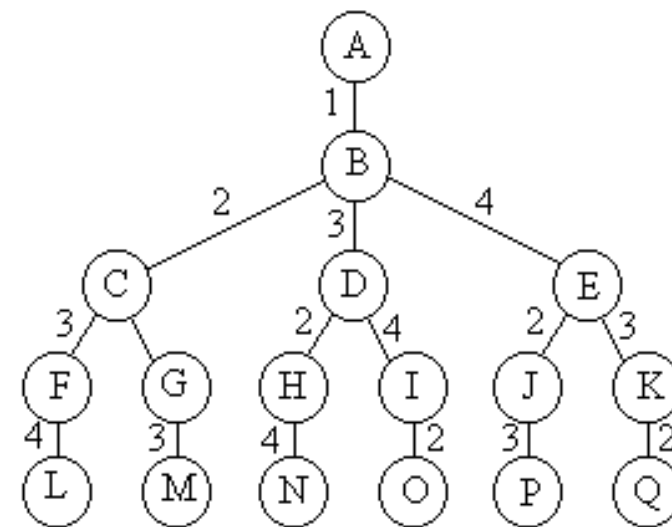
最小出边费用Minout

## 6.6 旅行售货员问题



### 2. 算法描述

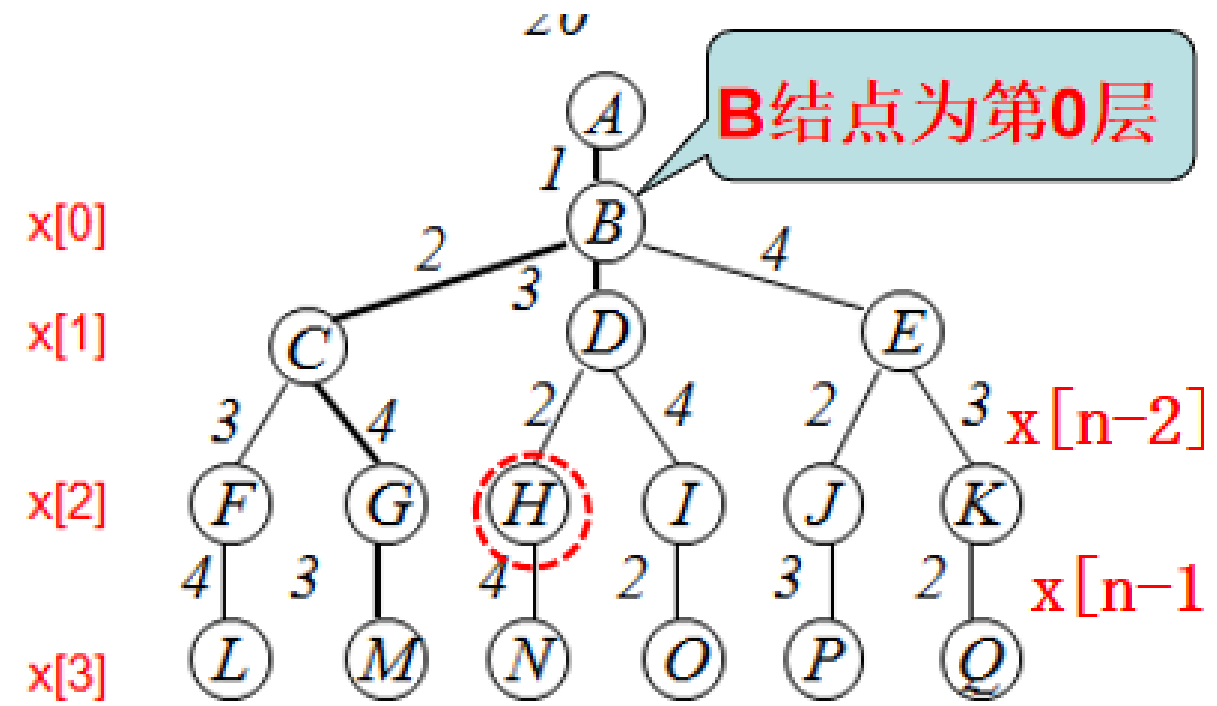
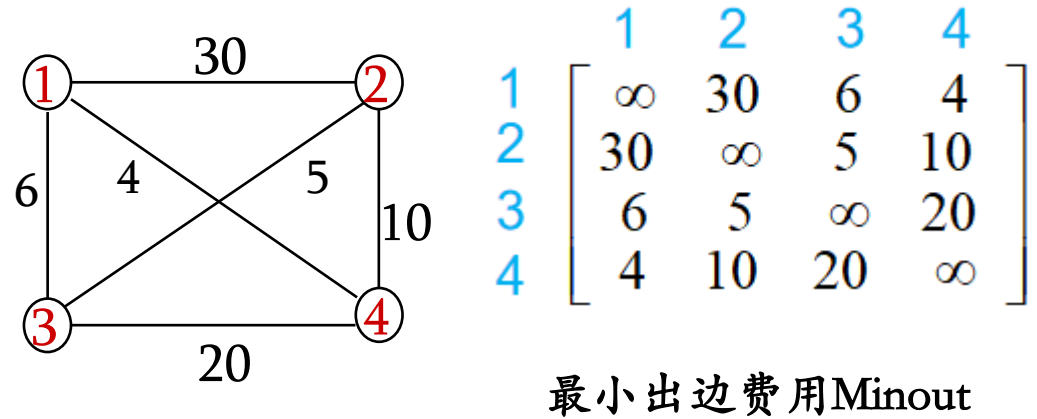
- 算法开始时创建一个最小堆，用于表示活结点优先队列。堆中每个结点的子树费用的下界lcost值是优先队列的优先级。接着算法计算出图中每个顶点的最小费用出边并用minout记录。如果所给的有向图中某个顶点没有出边，则该图不可能有回路，算法即告结束。如果每个顶点都有出边，则根据计算出的minout作算法初始化。
- 算法在搜索排序空间树的while循环体完成对排列树内部结点的扩展。对于当前扩展结点，算法分2种情况进行处理：



# 6.6 旅行售货员问题

## 2. 算法描述——搜索到树中某中间结点

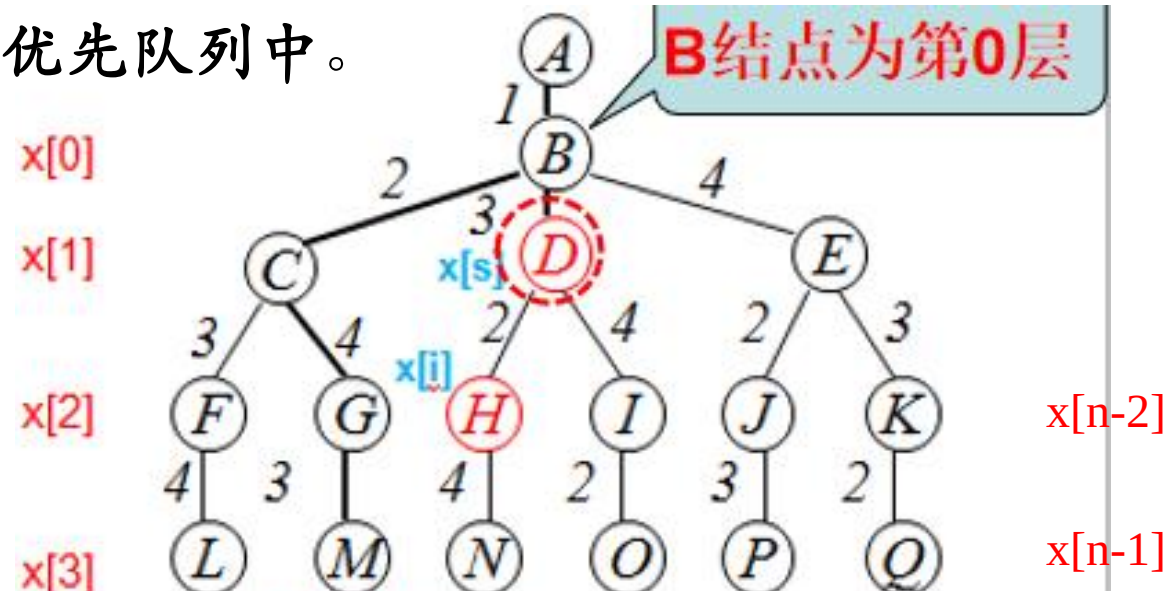
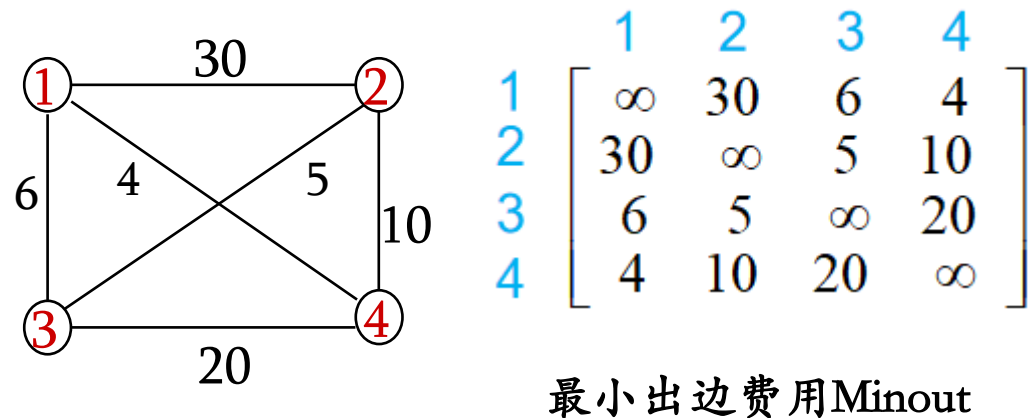
- 首先考虑 $s=n-2$ 的情形，此时当前扩展结点是排列树中某个叶结点的父结点。如果该叶结点对应一条可行回路且费用小于当前最小费用，则将该叶结点插入到优先队列中，否则舍去该叶结点。



# 6.6 旅行售货员问题

## 2. 算法描述——搜索到树中某中间结点

- 当 $s < n - 2$ 时，算法依次产生当前扩展结点的所有儿子结点。由于当前扩展结点所相应的路径是 $x[0:s]$ ，其可行儿子结点是从剩余顶点 $x[s+1:n-1]$ 中选取的顶点 $x[i]$ ，且 $(x[s], x[i])$ 是所给有向图 $G$ 中的一条边。对于当前扩展结点的每一个可行儿子结点，计算出其前缀 $(x[0:s], x[i])$ 的费用 $cc$ 和相应的下界 $lcost$ 。当 $lcost < bestc$ 时，将这个可行儿子结点插入到活结点优先队列中。



### 2. 算法描述

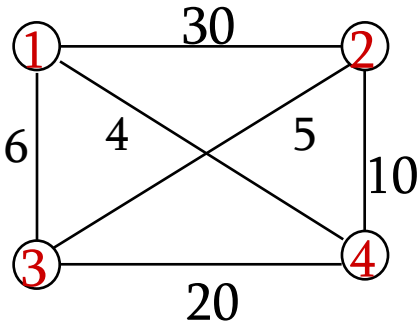
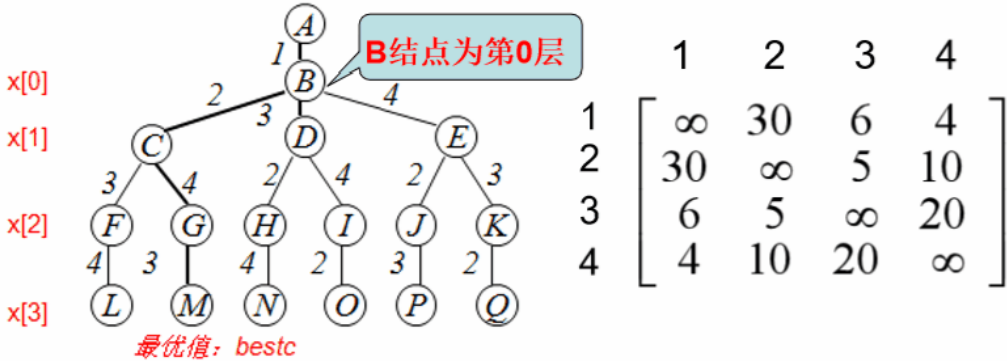
- 算法中while循环的终止条件是排列树的一个叶结点成为当前扩展结点。当 $s=n-1$ 时，已找到的回路前缀是 $x[0:n-1]$ ，它已包含图 $G$ 的所有 $n$ 个顶点。因此，当 $s=n-1$ 时，相应的扩展结点表示一个叶结点。此时该叶结点所相应的回路的费用等于 $cc$ 和 $lcost$ 的值。
- 剩余的活结点的 $lcost$ 值不小于已找到的回路的费用。它们都不可能导致费用更小的回路。因此已找到的叶结点所相应的回路是一个最小费用旅行售货员回路，算法可以结束。
- 算法结束时返回找到的最小费用，相应的最优解由数组 $v$ 给出。

# 6.6 旅行售货员问题

```
class MinHeapNode {
    Friend Traveling<Type>;
    Public:
        Operator Type () const { return lcost;}
    private:
        Type lcost, 费用下界
        Type cc, 当前费用
        Type rcost, 顶点最小出边费用和(顶点为: x[s:n-1])
        int s, 当前结点在排列树中的层次;
        Int *x, 存储解(结点路径)
}
```

bestc: 初始化为较大数值

下界:  $lcost=cc+rcost$



|   |          |          |          |          |
|---|----------|----------|----------|----------|
|   | 1        | 2        | 3        | 4        |
| 1 | $\infty$ | 30       | 6        | 4        |
| 2 | 30       | $\infty$ | 5        | 10       |
| 3 | 6        | 5        | $\infty$ | 20       |
| 4 | 4        | 10       | 20       | $\infty$ |

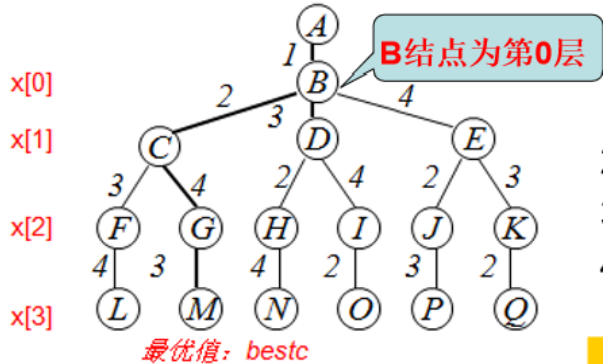
最小出边费用Minout

Minout[1]=4 Minout[2]=5  
Minout[3]=5 Minout[4]=4

# 6.6 旅行售货员问题

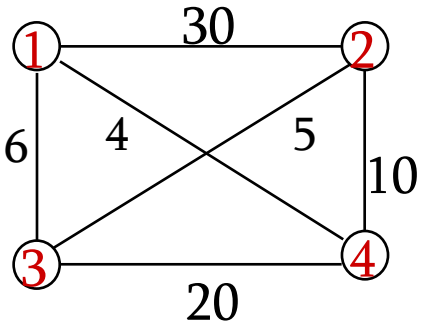
```
class MinHeapNode {
    Friend Traveling<Type>;
    Public:
        Operator Type () const { return lcost;}
    private:
        Type lcost, 费用下界
        Type cc, 当前费用
        Type rcost, 顶点最小出边费用和(顶点为: x[s:n-1])
        int s, 当前结点在排列树中的层次;
        Int *x, 存储解(结点路径)
}
```

bestc: 初始化为较大数值  
下界:  $lcost = cc + rcost$



|   | 1        | 2        | 3        | 4        |
|---|----------|----------|----------|----------|
| 1 | $\infty$ | 30       | 6        | 4        |
| 2 | 30       | $\infty$ | 5        | 10       |
| 3 | 6        | 5        | $\infty$ | 20       |
| 4 | 4        | 10       | 20       | $\infty$ |

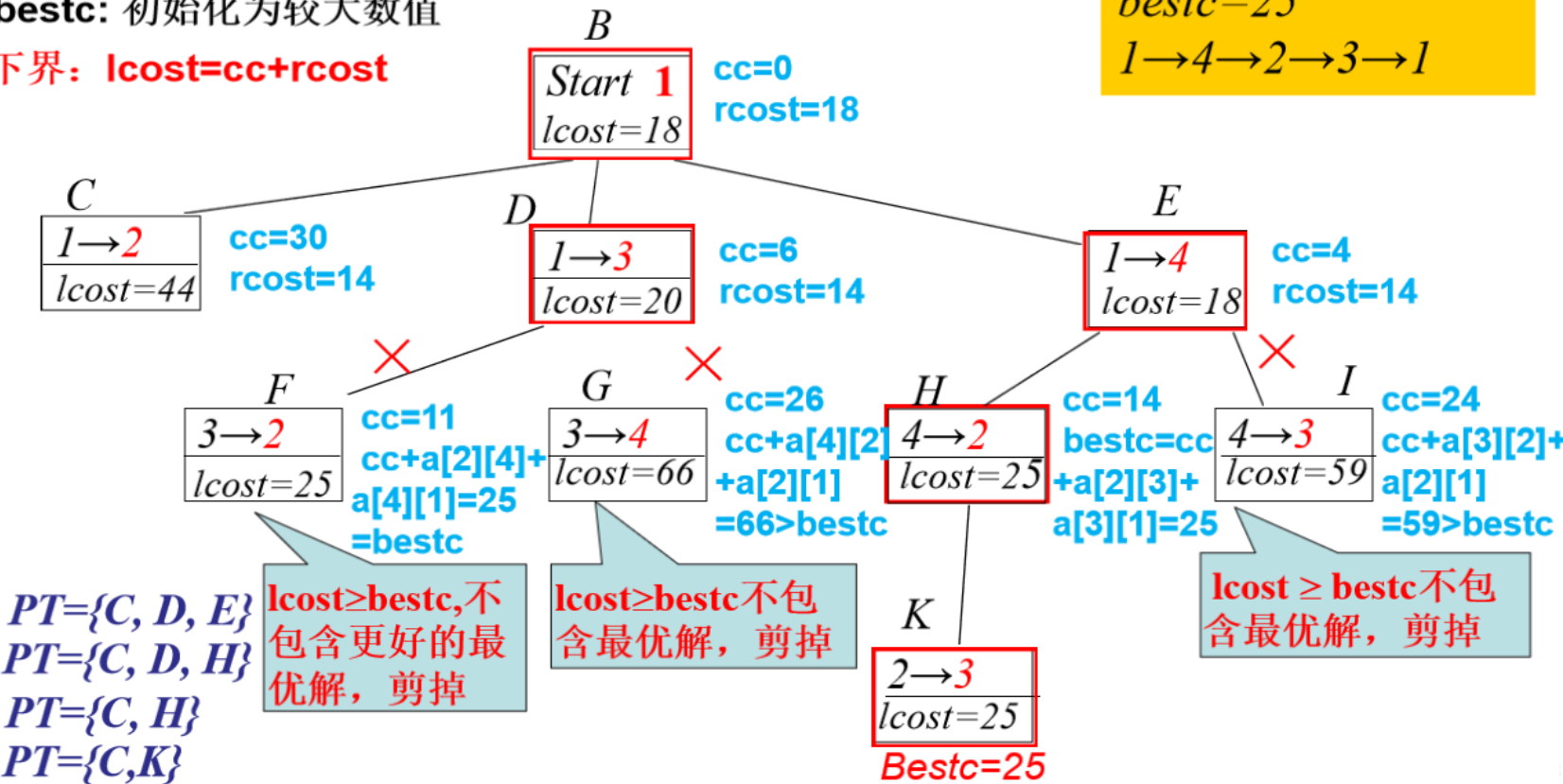
$bestc = 25$   
 $1 \rightarrow 4 \rightarrow 2 \rightarrow 3 \rightarrow 1$



|   | 1        | 2        | 3        | 4        |
|---|----------|----------|----------|----------|
| 1 | $\infty$ | 30       | 6        | 4        |
| 2 | 30       | $\infty$ | 5        | 10       |
| 3 | 6        | 5        | $\infty$ | 20       |
| 4 | 4        | 10       | 20       | $\infty$ |

最小出边费用Minout

Minout[1]=4 Minout[2]=5  
Minout[3]=5 Minout[4]=4



## 6.6 旅行售货员问题



### MinHeapNode

```
Class MinHeapNode{  
    Type lcost,    //子树费用下界  
    cc, //当前费用  
    rcost; //x[s:n-1]中顶点最小出边费用和  
    int s, //根节点到当前结点的路径x[0:s]  
    *x; //需要进一步搜索的顶点是x[s+1:n-1]  
}
```

## 6.6 旅行售货员问题



### 准备工作——计算MinOut

// 计算MinOut[i] = 顶点i的最小出边费用

```
Type *MinOut = new Type [n+1];
```

```
MinSum = 0; // 最小出边费用和
```

```
for (int i = 1; i <= n; i++) {
```

```
    Type Min = NoEdge;
```

```
    for (int j = 1; j <= n; j++)
```

```
        if (a[i][j] != NoEdge &&
```

```
            (a[i][j] < Min || Min == NoEdge))
```

```
                Min = a[i][j];
```

```
    if (Min == NoEdge) return NoEdge; // 此路不通
```

```
    MinOut[i] = Min;
```

```
    MinSum += Min; }
```

## 6.6 旅行售货员问题



### 准备工作——初始化

```
// 把E-节点初始化为树根
MinHeapNode E;
E.x = new int [n];
for (i = 0; i < n; i++)
    E.x[i] = i + 1;
E.s = 0; //根节点到当前节点的路径
E.cc = 0; // 当前费用为0
E.rcost = MinSum; //顶点最小出边费用和
bestc = NoEdge; // 目前没有找到旅行路径
```

```
Class MinHeapNode{
    Type lcost, //子树费用下界
    cc, //当前费用
    rcost; //x[s:n-1]中顶点最小出边费用和
    int s, //根节点到当前结点的路径x[0:s]
    *x; //需要进一步搜索的顶点是x[s+1:n-1]
}
```

## 6.6 旅行售货员问题



### 算法

```
//s = n-2 时:
while (E.s < n - 1) { //搜索排列树,不是叶子
    if (E.s == n - 2) { // 叶子的父节点, 再添加两条边即完成
        // 检查新的旅行路径是不是更好
        if (a[E.x[n-2]][E.x[n-1]] != NoEdge && a[E.x[n-1]][1]
            != NoEdge && (E.cc + a[E.x[n-2]][E.x[n-1]] +
                a[E.x[n-1]][1] < bestc || bestc == NoEdge))
            // 找到更优的旅行路径
            {bestc = E.cc + a[E.x[n-2]][E.x[n-1]] + a[E.x[n-1]][1];
            E.cc = bestc;
            E.lcost = bestc;
            E.s++;
            H . Insert(E); }
        else delete [] E.x;
    } //舍弃扩展结点
```

## 6.6 旅行售货员问题



### 算法

//s<n-2时:

else { // 产生孩子

for (int i = E.s + 1; i < n; i++)

if (a[E.x[E.s]][E.x[i]] != NoEdge) {

    // 可行的孩子, 限定了路径的耗费

    cc = E.cc + a[E.x[E.s]][E.x[i]];

    rcost = E.rcost - MinOut[E.x[E.s]]; // 剩下节点的

    b = cc + rcost; // 下限

    if (b < bestc || bestc == NoEdge) {

        // 子树可能有更好的叶子

        // 把根保存到堆中

## 6.6 旅行售货员问题



### 算法

```
MinHeapNode N;  
N.x = new int [n]; //新建立堆节点  
for (int j = 0; j < n; j++)  
    N.x[j] = E.x[j];  
N.x[E.s+1] = E.x[i];  
N.x [i]= E.x[E.s+1];  
N.cc = cc;  
N.s = E.s + 1;  
N.lcost = b;  
N.rcost = rcost;  
H. Insert(N) ;  
}  
} // 结束可行的孩子，释放空间  
delete [] E.x;  
}  
} // 对本节点的处理结束  
try {H.DeleteMin(E);} // 取下一个E-节点  
catch (OutOfBounds) {break;} // 没有未处理的节点
```

## 6.6 旅行售货员问题



### 结果处理

```
if (bestc == NoEdge) return NoEdge; // 没有旅行路径
```

```
// 将最优路径复制到v[1:n] 中
```

```
for (i = 0; i < n; i++)
```

```
    v[i+1] = E.x[i];
```

```
while (true) { // 释放最小堆中的所有节点
```

```
    delete [] E.x;
```

```
    try { H.DeleteMin(E); }
```

```
    catch (OutOfBounds) { break; }
```

```
}
```

```
return bestc;
```

## 6.7 批处理作业调度问题



- 给定 $n$ 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ 。每个作业必须先由机器1处理，然后由机器2处理。作业 $J_i$ 需要机器 $j$ 的处理时间为 $t_{ji}$ 。对于一个确定的作业调度，设 $F_{ji}$ 是作业 $i$ 在机器 $j$ 上完成处理的时间。所有作业在机器2上完成处理的时间和称为该作业调度的完成时间和。
$$f = \sum_{i=1}^n F_{2i}$$
- 批处理作业调度问题要求对于给定的 $n$ 个作业，制定最佳作业调度方案，使其完成时间和达到最小。

| $t_{ji}$ | 机器 1 | 机器 2 |
|----------|------|------|
| 作业 1     | 2    | 1    |
| 作业 2     | 3    | 1    |
| 作业 3     | 2    | 3    |

1,2,3

| $t_{ji}$ | 机器 1  | 机器 2   |
|----------|-------|--------|
| 作业1      | 2 (2) | 1 (3)  |
| 作业2      | 3 (5) | 1 (6)  |
| 作业3      | 2 (7) | 3 (10) |

1,3,2

| $t_{ji}$ | 机器 1  | 机器 2  |
|----------|-------|-------|
| 作业1      | 2 (2) | 1 (3) |
| 作业3      | 2 (4) | 3 (7) |
| 作业2      | 3 (7) | 1 (8) |

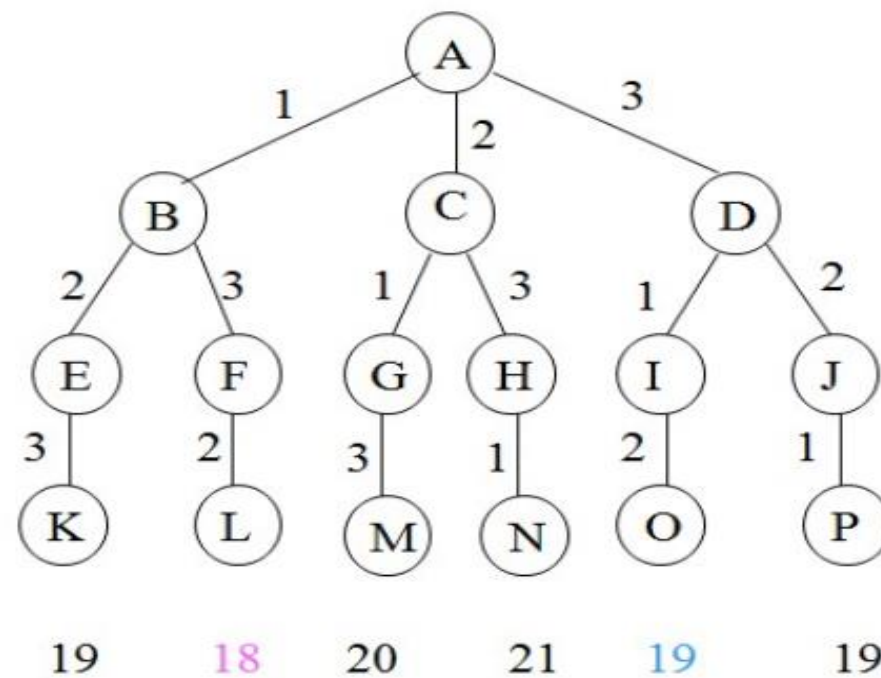
- 这3个作业的6种可能的调度方案是1,2,3; 1,3,2; 2,1,3; 2,3,1; 3,1,2; 3,2,1; 它们所相应的完成时间和分别是19, 18, 20, 21, 19, 19。易见，最佳调度方案是1,3,2，其完成时间和为18。

## 6.7 批处理作业调度问题



| $t_{ji}$ | 机器 1 | 机器 2 |
|----------|------|------|
| 作业 1     | 2    | 1    |
| 作业 2     | 3    | 1    |
| 作业 3     | 2    | 3    |

| $t_{ji}$ | 机器1  | 机器2   |
|----------|------|-------|
| 作业1      | 2(2) | 1(3)  |
| 作业2      | 3(5) | 1(6)  |
| 作业3      | 2(7) | 3(10) |



以1,2,3为例,

- 作业1在机器1上完成的时间为2, 在机器2上完成的时间为3
- 作业2在机器1上完成的时间为5, 在机器2上完成的时间为6
- 作业3在机器1上完成的时间为7, 在机器2上完成的时间为10
- 完成时间和=3+6+10=19

# 6.7 批处理作业调度问题



根节点至当前  
节点：已经安  
排的作业时间

当前节点至叶  
子节点：未安  
排的作业时间

## 2. 限界函数

- 在结点E处相应子树中叶结点完成时间和：
$$f = \sum_{i \in M} F_{2i} + \sum_{i \notin M} F_{2i}$$

机器1没有空闲  
时剩余任务的  
总完成时间和

- 剩余任务完成时间和的下界是：
$$\sum_{i \notin M} F_{2i} \geq \max\{S_1, S_2\}$$

机器2没有空闲时剩余  
任务的总完成时间和

- 设 $|M|=r$ , 注意到如果选择 $p_k$ , 使 $t_{1p_k}$ 在 $k \geq r+1$ 时依非减序排列,  $S_1$ 则取得极小值。  
同理如果选择 $p_k$ 使 $t_{2p_k}$ 依非减序排列, 则 $S_2$ 取得极小值。

$$f \geq \sum_{i \in M} F_{2i} + \max\{\hat{S}_1, \hat{S}_2\}$$

- 这可以作为优先队列式分支限界法中的限界函数。

## 6.7 批处理作业调度问题



### 3. 算法描述

- 算法的while循环完成对排列树内部结点的有序扩展。在while循环体内算法依次从活结点优先队列中取出具有最小bb值（完成时间和下界）的结点作为当前扩展结点，并加以扩展。
- 首先考虑 $E.s=n$ 的情形，当前扩展结点E是排列树中的叶结点。E.sf2是相应于该叶结点的完成时间和。当 $E.sf2 < bestc$ 时更新当前最优值bestc和相应的当前最优解bestx。当 $E.s < n$ 时，算法依次产生当前扩展结点E的所有儿子结点。对于当前扩展结点的每一个儿子结点node，计算出其相应的完成时间和的下界bb。当 $bb < bestc$ 时，将该儿子结点插入到活结点优先队列中。而当 $bb \geq bestc$ 时，可将结点node舍去。

## 6.7 批处理作业调度问题



### 3. 算法描述

当 $E.sf2 < bestc$ 时，更新当前最优值 $bestc$ 和相应的最优解 $bestx$

```
while (E.s <= n ) {  
    if (E.s == n ) {          // 叶结点  
        if (E.sf2 < bestc) {  
            bestc = E.sf2;  
            for (int i = 0; i < n; i++)  
                bestx[i] = E.x[i];  
            delete [] E.x;  
        }  
    }
```

```
else { // 产生当前扩展结点的儿子结点  
    for (int i = E.s; i < n; i++) {  
        Swap(E.x[E.s],E.x[i]);  
        int f1,f2;  
        int bb=Bound(E,f1,f2,y);  
        if (bb < bestc ) {  
            MinHeapNode N;  
            N.NewNode(E,f1,f2,bb,n);  
            H.Insert(N);  
            Swap(E.x[E.s],E.x[i]);  
        }  
    }  
    delete [] E.x; } // 完成结点扩展
```

当 $bb < bestc$ 时，将儿子结点插入到活结点优先队列中

- 分支限界法是在解空间树上的广度优先搜索算法。
- 分为队列式和优先队列式两种分支限界法。
- 和回溯法解决问题的类型相似，但解决问题的方法不同。

# 算法设计与分析

## 第8章 线性规划 (Linear Programming)



- 理解线性规划算法模型
- 掌握解线性规划问题的单纯形算法

# 8.1 线性规划问题



- 某企业计划生产甲、乙两种产品。这些产品分别要在A、B、C、D、四种不同的设备上加工。按工艺资料规定，单件产品在不同设备上加工所需要的台时如下表所示，企业决策者应如何安排生产计划，使企业总的利润最大？

| 产 品 \ 设 备 | A  | B | C  | D  | 利润 (元) |
|-----------|----|---|----|----|--------|
| 甲         | 2  | 1 | 4  | 0  | 2      |
| 乙         | 2  | 2 | 0  | 4  | 3      |
| 有效台时      | 12 | 8 | 16 | 12 |        |

设 $x_1$ 、 $x_2$ 分别为甲、乙两种产品的产量

$$\begin{aligned} \max \quad & Z = 2x_1 + 3x_2 \\ \text{s.t.} \quad & \begin{cases} 2x_1 + 2x_2 \leq 12 \\ x_1 + 2x_2 \leq 8 \\ 4x_1 \leq 16 \\ 4x_2 \leq 12 \\ x_1 \geq 0, x_2 \geq 0 \end{cases} \end{aligned}$$

# 8.1 线性规划问题



- 线性规划的数学模型由三个要素构成：

- 决策变量 Decision variables
- 目标函数 Objective function
- 约束条件 Constraints

$$\begin{aligned} \max \quad & Z = 2x_1 + 3x_2 \\ \text{s.t.} \quad & \begin{cases} 2x_1 + 2x_2 \leq 12 \\ x_1 + 2x_2 \leq 8 \\ 4x_1 \leq 16 \\ 4x_2 \leq 12 \\ x_1 \geq 0, x_2 \geq 0 \end{cases} \end{aligned}$$

- 怎样辨别一个模型是线性规划模型？

- 特征是：

- 问题的目标函数是多个决策变量的线性函数，通常是求最大值或最小值；
- 问题的约束条件是一组多个决策变量的线性不等式或等式。

# 8.1 线性规划问题



## 线性规划问题及其表示

- 线性规划问题可表示为如下形式:

$$\max \quad \sum_{j=1}^n c_j x_j \quad (8.1)$$

$$\text{s.t.} \quad \sum_{t=1}^n a_{it} x_t \leq b_i \quad i = 1, 2, \dots, m_1 \quad (8.2)$$

$$\sum_{t=1}^n a_{jt} x_t = b_j \quad j = m_1 + 1, \dots, m_1 + m_2 \quad (8.3)$$

$$\sum_{t=1}^n a_{kt} x_t \geq b_k \quad k = m_1 + m_2 + 1, \dots, m_1 + m_2 + m_3 \quad (8.4)$$

$$x_t \geq 0 \quad t = 1, 2, \dots, n \quad (8.5)$$

# 8.1 线性规划问题



- 变量满足约束条件(8.2)-(8.5)式的一组值称为线性规划问题的一个可行解。
- 所有可行解构成的集合称为线性规划问题的可行区域。
- 使目标函数取得极值的可行解称为最优解。
- 在最优解处目标函数的值称为最优值。
- 有些情况下可能不存在最优解。
- 通常有两种情况：
  - (1) 根本没有可行解，即给定的约束条件之间是相互排斥的，可行区域为空集；
  - (2) 目标函数没有极值，也就是说在 $n$ 维空间中的某个方向上，目标函数值可以无限增大，而仍满足约束条件，此时目标函数值无界。

# 8.1 线性规划问题



## 线性规划模型

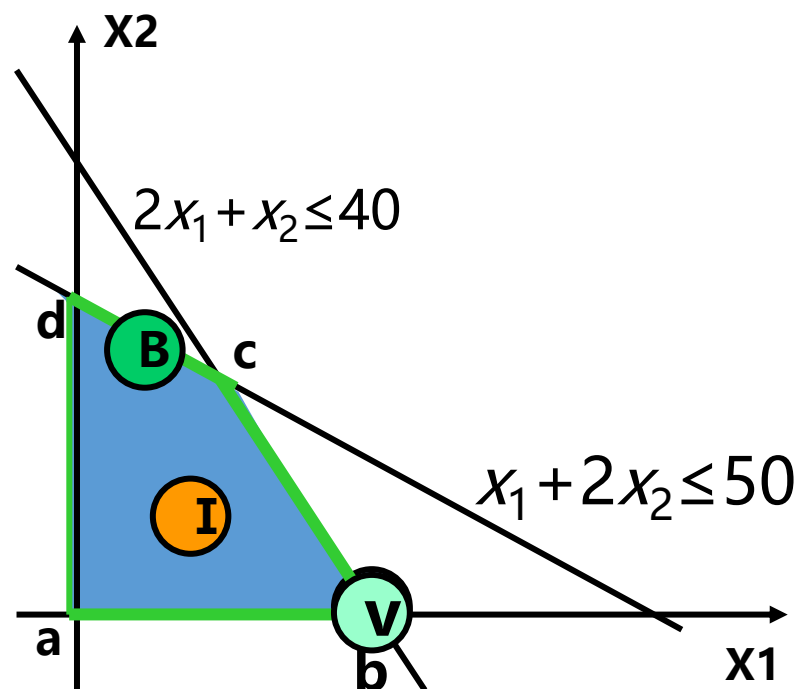
### ■ 线性规划解的若干概念:

$$\begin{array}{ll}\max & z = 5x_1 + 3x_2 \\ \text{s.t.} & 2x_1 + x_2 \leq 40 \\ & x_1 + 2x_2 \leq 50 \\ & x_1, x_2 \geq 0\end{array}$$

可行点

可行域

凸多面体

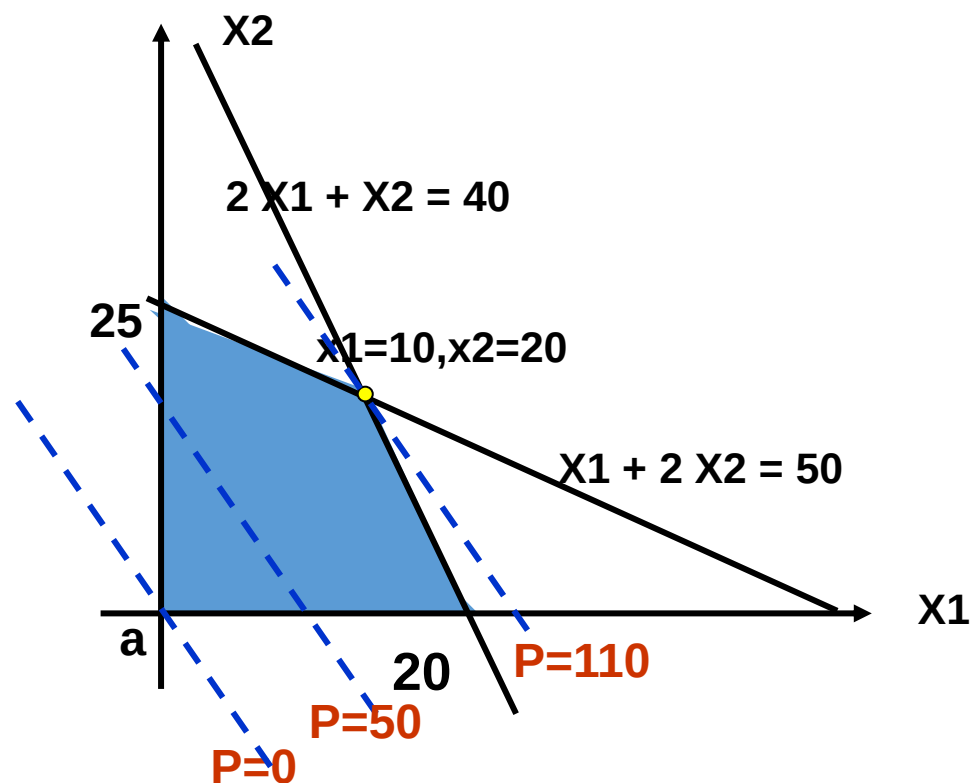


# 8.1 线性规划问题



## 线性规划模型

### ■ 线性规划解的图示：



$$\begin{aligned} \max \quad & z = 5x_1 + 3x_2 \\ \text{s.t.} \quad & 2x_1 + x_2 \leq 40 \\ & x_1 + 2x_2 \leq 50 \\ & x_1, x_2 \geq 0 \end{aligned}$$

# 8.1 线性规划问题

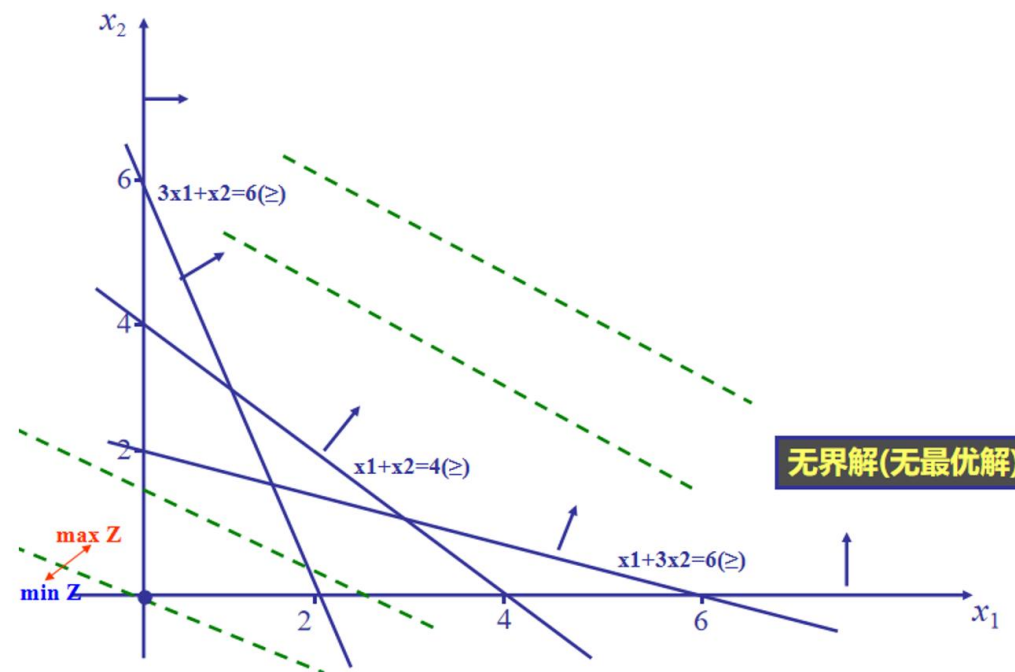


## 线性规划模型

### ■ 无最优解：

$$\max Z = x_1 + 2x_2$$

$$s.t = \begin{cases} x_1 + 3x_2 \geq 6 \\ x_1 + x_2 \geq 4 \\ 3x_1 + x_2 \geq 6 \\ x_1, x_2 \geq 0 \end{cases}$$



# 8.1 线性规划问题

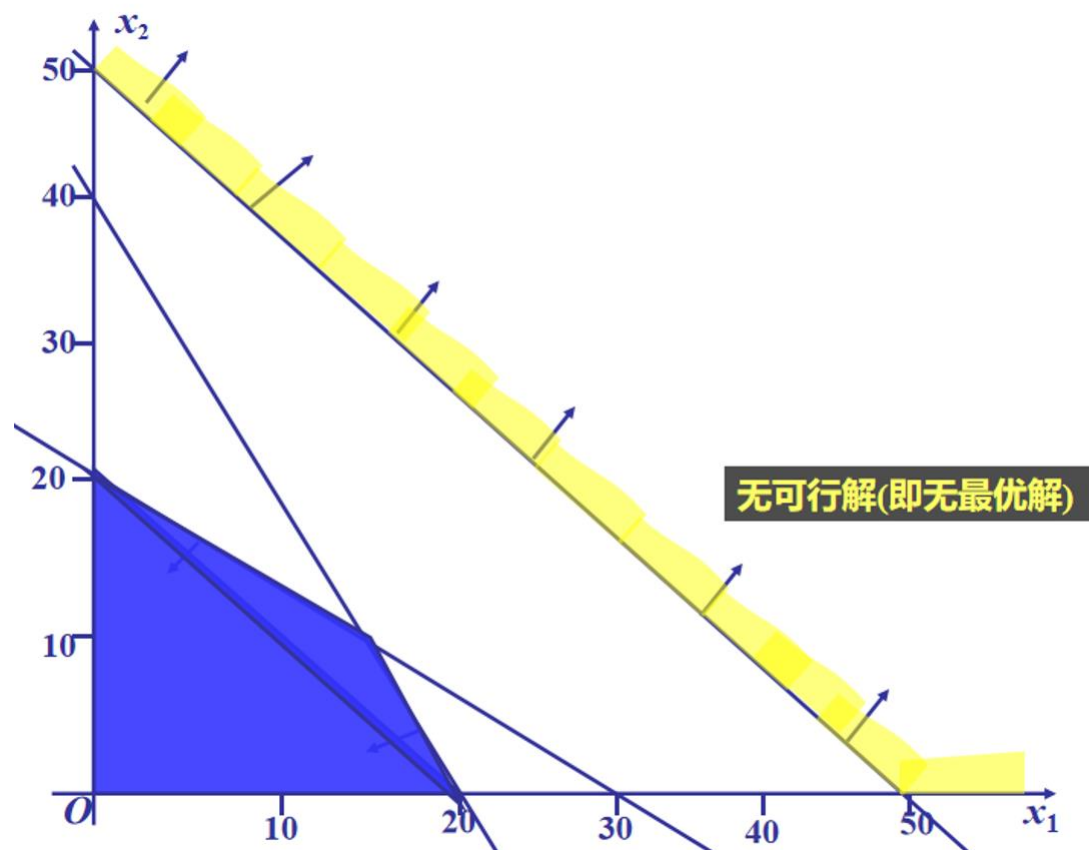


## 线性规划模型

### ■ 无可行解:

$$\max Z = 3x_1 + 4x_2$$

$$s.t = \begin{cases} 2x_1 + x_2 \leq 40 \\ x_1 + 1.5x_2 \leq 30 \\ x_1 + x_2 \geq 50 \\ x_1, x_2 \geq 0 \end{cases}$$

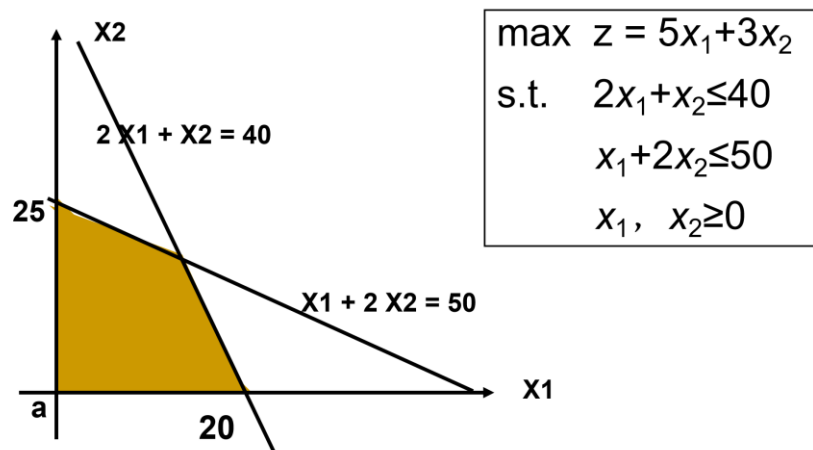


# 8.1 线性规划问题



## ■ 图解法

- 优点：可直观简便地获得最优解。
- 缺点：高维问题很难画图，同时如果没有代数语言的描述就缺乏严谨理论保证。
- 最优解非常容易出现在可行域构成的多面体的顶点处。
- 设想如果最优解总是会出现在顶点上的话，我们就可以只通过搜索可行域构成的多面体的顶点（极点）就可以找到最优解了，这个方法被称为单纯型法。



# 8.1 线性规划问题



## 线性规划基本定理

- 约束条件(8.2)-(8.5)  $n+m$  个约束中某  $n$  个约束以等号满足的可行解称为线性规划问题的基本可行解。
- 若  $n > m$ ，则基本可行解中至少有  $n-m$  个分量为0，也就是说，基本可行解中最多有  $m$  个分量非零。
- 线性规划基本定理：如果线性规划问题有最优解，则必有一基本可行最优解。
- 上述定理的重要意义在于，它把一个最优化问题转化为一个组合问题，即在(8.2)-(8.5)式的  $m+n$  个约束条件中，确定最优解应满足其中哪  $n$  个约束条件的问题。
- 由此可知，只要对各种不同的组合进行测试，并比较每种情况下的目标函数值，直到找到最优解。

## 8.2 线性规划问题的单纯形法



### 单纯形法

- Dantzig于1948年提出了线性规划问题的单纯形算法。
- 单纯形算法的特点是：
  - (1) 只对约束条件的若干组合进行测试，测试的每一步都使目标函数的值增加；
  - (2) 一般经过不大于 $m$ 或 $n$ 次迭代就可求得最优解。

## 8.2 线性规划问题的单纯形法



### 约束标准型线性规划问题的单纯形算法

- 当线性规划问题中 **没有不等式约束(8.2)和(8.4)式**，而只有等式约束(8.3)和变量非负约束(8.5)时，称该线性规划问题具有标准形式
- 先考察一类更特殊的标准形式线性规划问题。这一类线性规划问题中，**每一个等式约束中，至少有一个变量的系数为正**，且这个变量只在该约束中出现。
- 在每一约束方程中选择一个这样的变量，并以它作为变量求解该约束方程。这样选出来的变量称为左端变量或**基本变量**，其总数为 $m$ 个。剩下的 $n-m$ 个变量称为右端变量或**非基本变量**。
- 这一类特殊的标准形式线性规划问题称为**约束标准型线性规划问题**

## 8.2 线性规划问题的单纯形法



### 约束标准型线性规划问题的单纯形算法

- 虽然约束标准型线性规划问题非常特殊，但是对于理解线性规划问题的单纯形算法是非常重要的。
- 后面将看到，任意一个线性规划问题可以转换为约束标准型线性规划问题。

$$\begin{aligned} \max Z &= \sum_{j=1}^n c_j x_j \quad \text{-----(1)} \\ s.t \quad &\begin{cases} \sum_{j=1}^n a_{ij} x_j = b_i \quad (i=1,2,\dots,m) & \text{--(2)} \\ x_j \geq 0, j=1,2,\dots,n & \text{-----(3)} \end{cases} \end{aligned}$$

特点：

- (1) 目标函数求最大值（有时求最小值）
- (2) 约束条件都为等式方程，且右端常数项 $b_i$ 都大于或等于零
- (3) 决策变量 $x_j$ 为非负。

## 8.2 线性规划问题的单纯形法



### 示例

$$\max \quad z = -x_2 + 3x_3 - 2x_5$$

$$x_1 + 3x_2 - x_3 + 2x_5 = 7$$

$$x_4 - 2x_2 + 4x_3 = 12$$

$$x_6 - 4x_2 + 3x_3 + 8x_5 = 10$$

$$x_i \geq 0 \quad i = 1, 2, 3, 4, 5, 6$$

|       |    | $x_2$ | $x_3$ | $x_5$ |
|-------|----|-------|-------|-------|
| $z$   | 0  | -1    | 3     | -2    |
| $x_1$ | 7  | 3     | -1    | 2     |
| $x_4$ | 12 | -2    | 4     | 0     |
| $x_6$ | 10 | -4    | 3     | 8     |

## 8.2 线性规划问题的单纯形法



- 任何约束标准型线性规划问题，只要将所有非基本变量都置为0，从约束方程式中解出满足约束的基本变量的值，可求得一个基本可行解。
- 单纯形算法的基本思想就是从从一个基本可行解出发，进行一系列的基本可行解的变换。
- 每次变换将一个非基本变量与一个基本变量互调位置，且保持当前的线性规划问题是一个与原问题完全等价的标准线性规划问题。
- 基本可行解：  $x=(7,0,0,12,0,10)$ 。

|       |    | $x_2$ | $x_3$ | $x_5$ |
|-------|----|-------|-------|-------|
| $z$   | 0  | -1    | 3     | -2    |
| $x_1$ | 7  | 3     | -1    | 2     |
| $x_4$ | 12 | -2    | 4     | 0     |
| $x_6$ | 10 | -4    | 3     | 8     |

## 8.2 线性规划问题的单纯形法



- 单纯形算法的第1步：选出使目标函数增加的非基本变量作为入基变量。
- 查看单纯形表的第1行（也称之为z行）中标有非基本变量的各列中的值。
- 选出使目标函数增加的非基本变量作为入基变量。
- z行中的正系数非基本变量都满足要求。
- 在上面单纯形表的z行中只有1列为正，即非基本变量相应的列，其值为3。
- 选取非基本变量 $x_3$ 作为入基变量。

|       |    | $x_2$ | $x_3$ | $x_5$ |
|-------|----|-------|-------|-------|
| z     | 0  | -1    | 3     | -2    |
| $x_1$ | 7  | 3     | -1    | 2     |
| $x_4$ | 12 | -2    | 4     | 0     |
| $x_6$ | 10 | -4    | 3     | 8     |

## 8.2 线性规划问题的单纯形法



- 单纯形算法的第2步：选取离基变量。
- 在单纯形表中考察由第1步选出的入基变量所相应的列。
- 在一个基本变量变为负值之前，入基变量可以增到多大。
- 如果入基变量所在的列与基本变量所在行交叉处的表元素为负数，那么该元素将不受任何限制，相应的基本变量只会越变越大。
- 如果入基变量所在列的所有元素都是负值，则目标函数无界，已经得到了问题的无界解。
- 如果选出的列中有一个或多个元素为正数，要弄清是哪个数限制了入基变量值的增加。

|       |    | $x_2$ | $x_3$ | $x_5$ |
|-------|----|-------|-------|-------|
| $z$   | 0  | -1    | 3     | -2    |
| $x_1$ | 7  | 3     | -1    | 2     |
| $x_4$ | 12 | -2    | 4     | 0     |
| $x_6$ | 10 | -4    | 3     | 8     |

## 8.2 线性规划问题的单纯形法



- 受限的增加量可以用入基变量所在列的元素（称为主元素）来除主元素所在行的“常数列”（最左边的列）中元素而得到。所得数值越小说明受到限制越多。
- 应该选取受到限制最多的基本变量作为离基变量，才能保证将入基变量与离基变量互调位置后，仍满足约束条件。
- 上例中，唯一的一个值为正的z行元素是3，它所在列中有2个正元素，即4和3。
- $\min\{12/4, 10/3\}=3$ ，应该选取 $x_4$ 为离基变量；入基变量 $x_3$ 取值为3。

|       |    | $x_2$ | $x_3$ | $x_5$ |
|-------|----|-------|-------|-------|
| $z$   | 0  | -1    | 3     | -2    |
| $x_1$ | 7  | 3     | -1    | 2     |
| $x_4$ | 12 | -2    | 4     | 0     |
| $x_6$ | 10 | -4    | 3     | 8     |

## 8.2 线性规划问题的单纯形法



- 单纯形算法的第3步：转轴变换。
- 转轴变换的目的是将入基变量与离基变量互调位置。
- 给入基变量一个增值，使之成为基本变量；
- 修改离基变量，让入基变量所在列中，离基变量所在行的元素值减为零，而使之成为非基本变量。

## 8.2 线性规划问题的单纯形法



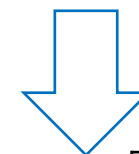
- 解离基变量所相应的方程，将入基变量 $x_3$ 用离基变量 $x_4$ 表示为：
$$x_3 - \frac{1}{2}x_2 + \frac{1}{4}x_4 = 3$$
- 再将其代入其他基本变量和所在的行中消去 $x_3$ :

$$x_1 + \frac{5}{2}x_2 + \frac{1}{4}x_4 + 2x_5 = 10$$

$$x_6 - \frac{5}{2}x_2 - \frac{3}{4}x_4 + 8x_5 = 1$$

- 代入目标函数得到：
$$z = 9 + \frac{1}{2}x_2 - \frac{3}{4}x_4 - 2x_5$$
- 形成新单纯形表

|       |    | $x_2$ | $x_3$ | $x_5$ |
|-------|----|-------|-------|-------|
| $z$   | 0  | -1    | 3     | -2    |
| $x_1$ | 7  | 3     | -1    | 2     |
| $x_4$ | 12 | -2    | 4     | 0     |
| $x_6$ | 10 | -4    | 3     | 8     |



|       |    | $x_2$ | $x_4$ | $x_5$ |
|-------|----|-------|-------|-------|
| $z$   | 9  | 1/2   | -3/4  | -2    |
| $x_1$ | 10 | 5/2   | 1/4   | 2     |
| $x_3$ | 3  | -1/2  | 1/4   | 0     |
| $x_6$ | 1  | -5/2  | -3/4  | 8     |

## 8.2 线性规划问题的单纯形法



- 单纯形算法的第4步：转回并重复第1步，进一步改进目标函数值。
- 不断重复上述过程，直到z行的所有非基本变量系数都变成负值为止。这表明目标函数不可能再增加了。
- 在上面的单纯形表中，唯一的值为正的z行元素是非基本变量 $x_2$ 相应的列，其值为 $1/2$ 。因此，选取非基本变量 $x_2$ 作为入基变量。
- 它所在列中有唯一的正元素 $5/2$ ，即基本变量 $x_1$ 相应行的元素。因此，选取 $x_1$ 为离基变量。

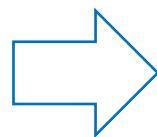
|       |    | $x_2$  | $x_4$  | $x_5$ |
|-------|----|--------|--------|-------|
| $z$   | 9  | $1/2$  | $-3/4$ | $-2$  |
| $x_1$ | 10 | $5/2$  | $1/4$  | 2     |
| $x_3$ | 3  | $-1/2$ | $1/4$  | 0     |
| $x_6$ | 1  | $-5/2$ | $-3/4$ | 8     |

## 8.2 线性规划问题的单纯形法



- 再经步骤3的转轴变换得到新单纯形表。
- 新单纯形表z行的所有非基本变量系数都变成负值，求解过程结束。
- 整个问题的解可以从最后一张单纯形表的常数列中读出。
- 目标函数的最大值为11；
- 最优解为： $x^*=(0,4,5,0,0,11)$ 。

|       |    | $x_2$ | $x_4$ | $x_5$ |
|-------|----|-------|-------|-------|
| z     | 9  | 1/2   | -3/4  | -2    |
| $x_1$ | 10 | 5/2   | 1/4   | 2     |
| $x_3$ | 3  | -1/2  | 1/4   | 0     |
| $x_6$ | 1  | -5/2  | -3/4  | 8     |



|       |    | $x_1$ | $x_4$ | $x_5$ |
|-------|----|-------|-------|-------|
| z     | 11 | -1/5  | -4/5  | -12/5 |
| $x_2$ | 4  | 2/5   | 1/10  | 4/5   |
| $x_3$ | 5  | 1/5   | 3/10  | 2/5   |
| $x_6$ | 11 | 1     | -1/2  | 10    |

## 8.2 线性规划问题的单纯形法



### 单纯形算法计算步骤

- 单纯形算法的计算过程可以用单纯形表的形式归纳为一系列基本矩阵运算。
- 主要运算为转轴变换，该变换类似解线性方程组的高斯消去法中的消元变换。

#### ■ 单纯形表：

- $x_1, x_2, \dots, x_m$  为基本变量， $x_{m+1}, x_{m+2}, \dots, x_n$  为非基本变量。
- 基本变量下标集为  $B = \{1, 2, \dots, m\}$ ;  
非基本变量下标集为  $N = \{m+1, m+2, \dots, n\}$ ;
- 当前基本可行解为  $(b_1, b_2, \dots, b_m, 0, \dots, 0)$ 。

|          |          | $x_{m+1}$  | $x_{m+2}$  | $\dots$ | $x_n$    |
|----------|----------|------------|------------|---------|----------|
| $Z$      | $c_0$    | $c_{m+1}$  | $c_{m+2}$  | $\dots$ | $c_n$    |
| $x_1$    | $b_1$    | $a_{1m+1}$ | $a_{1m+2}$ | $\dots$ | $a_{1n}$ |
| $x_2$    | $b_2$    | $a_{2m+1}$ | $a_{2m+2}$ | $\dots$ | $a_{2n}$ |
| $\vdots$ | $\vdots$ |            |            |         |          |
| $x_m$    | $b_m$    | $a_{mm+1}$ | $a_{mm+2}$ | $\dots$ | $a_{mn}$ |

## 8.2 线性规划问题的单纯形法



### 单纯形算法计算步骤

#### ■ 步骤1：选入基变量。

- 如果所有  $c_j \leq 0$ ，则当前基本可行解为最优解，计算结束。
- 否则取  $c_e > 0$  相应的非基本变量  $x_e$  为入基变量。

#### ■ 步骤2：选离基变量。

- 对于步骤1选出的入基变量  $x_e$ ，如果所有  $a_{ie} \leq 0$ ，则最优解无界，计算结束；

$$\text{否则，计算 } \theta = \min_{a_{ie} > 0} \left\{ \frac{b_i}{a_{ie}} \right\} = \frac{b_k}{a_{ke}}$$

- 选取基本变量  $x_k$  为离基变量。
- 新的基本变量下标集为  $\bar{B} = B + \{e\} - \{k\}$
- 新的非基本变量下标集为  $\bar{N} = N + \{k\} - \{e\}$

|          |          | $x_{m+1}$  | $x_{m+2}$  | ... | $x_n$    |
|----------|----------|------------|------------|-----|----------|
| $Z$      | $c_0$    | $c_{m+1}$  | $c_{m+2}$  | ... | $c_n$    |
| $x_1$    | $b_1$    | $a_{1m+1}$ | $a_{1m+2}$ | ... | $a_{1n}$ |
| $x_2$    | $b_2$    | $a_{2m+1}$ | $a_{2m+2}$ | ... | $a_{2n}$ |
| $\vdots$ | $\vdots$ |            |            |     |          |
| $x_m$    | $b_m$    | $a_{mm+1}$ | $a_{mm+2}$ | ... | $a_{mn}$ |

## 8.2 线性规划问题的单纯形法



### 单纯形算法计算步骤

■ 步骤3: 转轴变换。

■ 新单纯形表中各元素变换如下。

$$\begin{cases} \bar{b}_i = b_i - a_{ie} \frac{b_k}{a_{ke}} & i \in \bar{B} \\ \bar{b}_e = \frac{b_k}{a_{ke}} \end{cases} \quad \begin{cases} \bar{a}_{ij} = a_{ij} - a_{ie} \frac{a_{kj}}{a_{ke}} & i \in \bar{B}, \quad j \in \bar{N} \\ \bar{a}_{ik} = -\frac{a_{ie}}{a_{ke}} \end{cases}$$
$$\begin{cases} \bar{a}_{ej} = \frac{a_{kj}}{a_{ke}} & j \in \bar{N} \\ \bar{a}_{ek} = \frac{1}{a_{ke}} \end{cases} \quad \begin{cases} \bar{c}_i = c_i - c_e \frac{a_{ki}}{a_{ke}} & i \in \bar{N} \\ \bar{c}_k = -\frac{c_e}{a_{ke}} \end{cases}$$

■ 步骤4: 转步骤1。

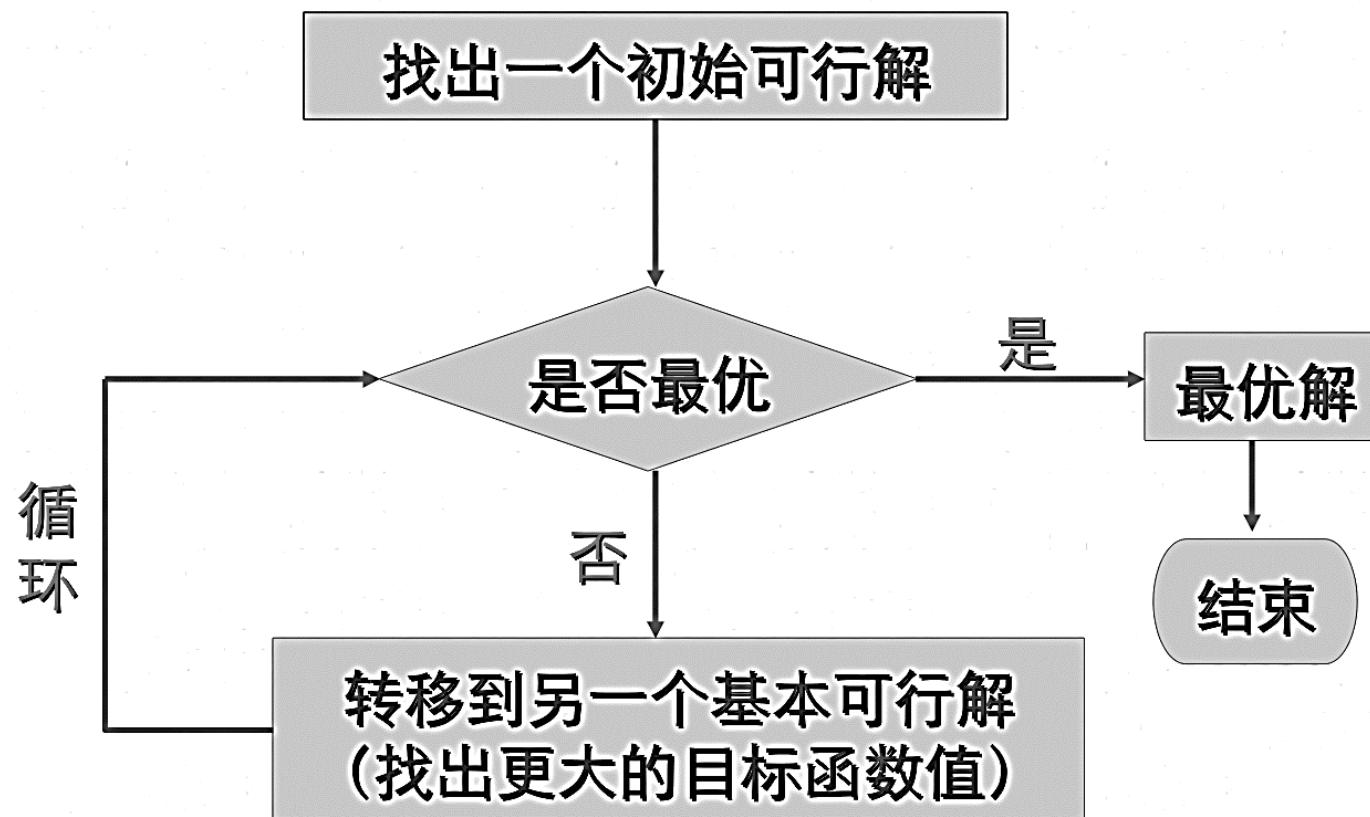
■ 新的基本变量下标集为  $\bar{B} = B + \{e\} - \{k\}$

■ 新的非基本变量下标集为  $\bar{N} = N + \{k\} - \{e\}$

## 8.2 线性规划问题的单纯形法



### 单纯形法思路



核心是：变量迭代

## 8.2 线性规划问题的单纯形法



### 将一般问题转化为约束标准型

#### ■ 线性规划标准形式

$$\max Z = \sum_{j=1}^n c_j x_j \quad \text{-----(1)}$$

$$s.t \begin{cases} \sum_{j=1}^n a_{ij} x_j = b_i \quad (i = 1, 2, \dots, m) & \text{--(2)} \\ x_j \geq 0, j = 1, 2, \dots, n & \text{-----(3)} \end{cases}$$

#### ■ 目标函数的转换

如果是求极小值即  $\min z = \sum c_j x_j$ ，则可将目标函数乘以(-1)，可化为求极大值问题。

即  $\max z' = -z = -\sum c_j x_j$

也就是：令  $z' = -z$ ，可得到上式。

## 8.2 线性规划问题的单纯形法



### 将一般问题转化为约束标准型

- 有几种巧妙的办法可以将一般的线性规划问题转换为约束标准型线性规划问题。
- 首先，需要把(8.2)或(8.4)形式的不等式约束转换为等式约束。
- 具体做法是，引入松弛变量，利用松弛变量的非负性，将不等式转化为等式。
- 松弛变量记为 $y_i$ ，共有 $m_1+m_3$ 个。
- 在求解过程中，应当将松弛变量与原来变量同样对待。求解结束后，抛弃松弛变量。
- 注意松弛变量前的符号由相应的原不等式的方向所确定。

$$\begin{array}{rcl} x_1 + 2x_3 & \leq & 18 \\ 2x_2 - 7x_4 & \leq & 0 \\ x_1 + x_2 + x_3 + x_4 & = & 9 \\ x_2 - x_3 + 2x_4 & \geq & 1 \end{array} \quad \Rightarrow \quad \begin{array}{rcl} x_1 + 2x_3 + y_1 & = & 18 \\ 2x_2 - 7x_4 + y_2 & = & 0 \\ x_1 + x_2 + x_3 + x_4 & = & 9 \\ x_2 - x_3 + 2x_4 - y_3 & = & 1 \end{array}$$

## 8.2 线性规划问题的单纯形法



### 进一步讨论——人工变量法

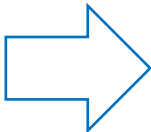
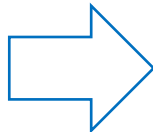
- 前面讨论了在标准型中系数矩阵有单位矩阵，很容易确定一组基可行解。在实际问题中有些模型并不含有单位矩阵，为了得到一组基向量和初基可行解，在约束条件的等式左端加一组虚拟变量，得到一组基变量。这种人为加的变量称为人工变量，构成的可行基称为人工基，用大M法或两阶段法求解，这种用人工变量作桥梁的求解方法称为人工变量法。

## 8.2 线性规划问题的单纯形法



### 进一步讨论——人工变量法

- 为了进一步构造标准型约束，还需要引入 $m$ 个人工变量，记为 $z_i$ 。
- 至此，原问题已经变换为约束标准型线性规划问题。
- 对极小化线性规划问题，只要将目标函数乘以-1即可化为等价的极大化线性规划问题。

|                             |                                                                                     |                              |                                                                                       |                                    |
|-----------------------------|-------------------------------------------------------------------------------------|------------------------------|---------------------------------------------------------------------------------------|------------------------------------|
| $x_1 + 2x_3 \leq 18$        |                                                                                     | $x_1 + 2x_3 + y_1 = 18$      |                                                                                       | $z_1 + x_1 + 2x_3 + y_1 = 18$      |
| $2x_2 - 7x_4 \leq 0$        |                                                                                     | $2x_2 - 7x_4 + y_2 = 0$      |                                                                                       | $z_2 + 2x_2 - 7x_4 + y_2 = 0$      |
| $x_1 + x_2 + x_3 + x_4 = 9$ |  | $x_1 + x_2 + x_3 + x_4 = 9$  |  | $z_3 + x_1 + x_2 + x_3 + x_4 = 9$  |
| $x_2 - x_3 + 2x_4 \geq 1$   |                                                                                     | $x_2 - x_3 + 2x_4 - y_3 = 1$ |                                                                                       | $z_4 + x_2 - x_3 + 2x_4 - y_3 = 1$ |

## 8.2 线性规划问题的单纯形法



### 一般线性规划问题的2阶段单纯形算法

- 引入人工变量后的线性规划问题与原问题并不等价，除非所有 $z_i$ 都是0。
- 为了解决这个问题，在求解时必须分2个阶段进行。
- 第一阶段用一个辅助目标函数  $z' = -\sum_{i=1}^m z_i$  替代原来的目标函数。
- 这个线性规划问题称为原线性规划问题所相应的辅助线性规划问题。
- 对辅助线性规划问题用单纯形算法求解。
- 如果原线性规划问题有可行解，则辅助线性规划问题就有最优解，且其最优值为0，即所有 $z_i$ 都为0。

## 8.2 线性规划问题的单纯形法



### 一般线性规划问题的2阶段单纯形算法

- 在辅助线性规划问题最后的单纯形表中，所有 $z_i$ 均为非基本变量。
- 划掉所有 $z_i$ 相应的列，剩下的就是只含 $x_i$ 和 $y_i$ 的约束标准型线性规划问题了。
- 单纯形算法第一阶段的任务就是构造一个初始基本可行解。
- 单纯形算法第二阶段的目标是求解由第一阶段导出的问题。
- 此时要用原来的目标函数进行求解。
- 如果在辅助线性规划问题最后的单纯形表中， $z_i$ 不全为0，则原线性规划问题没有可行解，从而原线性规划问题无解。

## 8.2 线性规划问题的单纯形法——



### 一般线性规划问题的单纯形算法

$$\max Z = 3x_1 + 2x_2 - x_3$$

$$\begin{cases} -4x_1 + 3x_2 + x_3 \geq 4 \\ x_1 - x_2 + 2x_3 \leq 10 \\ -2x_1 + 2x_2 - x_3 = -1 \\ x_1, x_2, x_3 \geq 0 \end{cases}$$

标准形式

$$\max Z = 3x_1 + 2x_2 - x_3$$

$$\begin{cases} -4x_1 + 3x_2 + x_3 - x_4 = 4 \\ x_1 - x_2 + 2x_3 + x_5 = 10 \\ 2x_1 - 2x_2 + x_3 = 1 \\ x_j \geq 0, j = 1, 2, \dots, 5 \end{cases}$$

系数矩阵中不存在单位矩阵，  
无法建立初始单纯形表。

人为添加两个单位向量，  
得到人工变量单纯形法

M是一个很大的抽象的数，不需要给出具体的数值，可以理解为它能大于给定的任何一个确定数值。再用前面介绍的单纯形法求解该模型。

$$\max Z' = -Mx_6 - Mx_7$$

$$\max Z = 3x_1 + 2x_2 - x_3$$

$$\begin{cases} -4x_1 + 3x_2 + x_3 - x_4 + x_6 = 4 \\ x_1 - x_2 + 2x_3 + x_5 = 10 \\ 2x_1 - 2x_2 + x_3 + x_7 = 1 \\ x_j \geq 0, j = 1, 2, \dots, 7 \end{cases}$$